

Research Engineering OS

把返工压缩成规范 + 模板 + 检查清单

Research Engineering OS

把返工压缩成规范 + 模板 + 检查清单

Copyright © 2026 Li Hongmin (李鸿敏)

All rights reserved.

Author: Li Hongmin (李鸿敏)

Department of Computational Biology and Medical Sciences

Graduate School of Frontier Sciences

The University of Tokyo

Contact: lihongmin@edu.k.u-tokyo.ac.jp

Website: li-hongmin.github.io

First Edition: February 2026

Contents

前言	iii
1 你为什么总在最后推翻（探索债 / 验证债 / 复现债）	1
1.1 故事引入：离截稿只剩几天，你突然不敢相信自己的结果	1
1.2 为什么会这样：三类债务在后期集中爆炸	1
1.3 症状表：你可能正在走向最后推翻	2
1.4 一个常见剧情：为什么”最后阶段爆炸”几乎必然发生	3
1.5 解释模型：三种债务	3
1.5.1 探索债（Exploration Debt）	4
1.5.2 验证债（Validation Debt）	4
1.5.3 复现债（Reproducibility Debt）	5
1.6 本章结论：默认行为，而非复杂工具	5
1.7 本书原则（贯穿后续章节的理念）	6
1.8 你现在就能做的 10 分钟动作	6
2 项目的最小单位不是”代码”，是”实验”	8

2.1	实验的定义：把”我改了点东西”变成可比的对象	8
2.1.1	必须能回答的 6 个问题	9
2.2	实验对象模型：将研究过程拆解成稳定的”五要素”	10
2.3	run_id：让每一次运行都可以被清晰指代	12
2.4	一条实用原则：将”可比性”放在第一优先级	13
3	仓库结构就是你的第二大脑	15
3.1	真实案例：快速堆代码的代价	15
3.2	一个可直接照抄的目录结构（研究友好）	17
3.3	快变量与慢变量：把”稳定”从”探索”里分离出来	17
3.4	每个目录的 Definition of Done (DoD)	18
3.4.1	src/：核心库（可复用、可测试）	18
3.4.2	experiments/：实验入口（可丢弃）	18
3.4.3	configs/：配置（可追溯）	19
3.4.4	outputs/：产物（可清理/可归档）	19
3.4.5	reports/：图表与结论（可再生成）	19
3.4.6	tests/：测试（守住底线）	19
3.5	入口收敛：让”怎么跑”变得显眼	20
3.6	渐进式重构：从混乱项目迁移到良好结构	20
3.6.1	第一步：识别并区分慢变量与快变量	20
3.6.2	第二步：提取配置与参数	21
3.6.3	第三步：统一输出路径	21

3.6.4	第四步：添加基本测试	22
3.7	多任务/多项目的目录层级管理	22
3.7.1	原则：优先考虑独立仓库	22
3.7.2	单仓库多项目的分层结构	23
3.7.3	命名约定和环境隔离	23
3.7.4	何时共享代码，何时复制代码	24
3.8	10 分钟动作：把当前项目” 分层一次”	24
4	Git 不是用来” 保存代码”，是用来” 证明历史”	26
4.1	故事引入：审稿意见要求复现，你却找不回当时的 代码	26
4.2	为什么” 随手 commit” 救不了你	27
4.3	研究中的 Git 使用陷阱	27
4.3.1	陷阱 1：commit 粒度太大，找不到关键改动	27
4.3.2	陷阱 2：实验和代码改动时间错位	28
4.3.3	陷阱 3：分支使用不当，主线混乱	29
4.4	适合研究的 Git 分支策略	29
4.4.1	推荐的分支结构	29
4.4.2	典型工作流程	30
4.5	用 Tag 标记里程碑：让论文结果永久可追溯	31
4.5.1	推荐的 Tag 命名规范	32
4.5.2	Tag 使用实践	32

4.6	实验产物不进 Git：用.gitignore 保持仓库干净	33
4.6.1	什么不应该提交到 Git	33
4.6.2	推荐的.gitignore 模板	34
4.7	常见问题与解决方案	35
4.7.1	Q1：代码已经改了很多，怎么补救？	35
4.7.2	Q2：团队协作时如何统一分支策略？	36
4.7.3	Q3：如何处理”dirty”状态的实验？	36
4.8	实战案例：从混乱到清晰的 Git 历史	37
4.8.1	重构前（反面教材）	37
4.8.2	重构后（正面示例）	37
4.9	10 分钟动作：为当前项目建立 Git 基线	38
5	从”能跑”到”可信”只差一个 Definition of Done	40
5.1	故事引入：那些”差不多就行”的代价	40
5.2	为什么”能跑”不等于”完成”	41
5.2.1	埋雷 1：”跑出结果”但无法复现	41
5.2.2	埋雷 2：”改进有效”但不知道为什么	41
5.2.3	埋雷 3：”对比实验”但评估口径不一致 . .	42
5.3	论文候选结论的 DoD 清单	42
5.3.1	最小 DoD（5 项必须满足）	42
5.3.2	增强 DoD（推荐额外满足）	44
5.4	DoD 检查清单：从”能跑”到”可信”的操作步骤 .	46

5.4.1	实验完成后立即做（5 分钟）	46
5.4.2	准备写论文前做（30 分钟）	46
5.4.3	论文提交前做（1 小时）	48
5.5	团队如何使用 DoD	49
5.5.1	作为合并标准	49
5.5.2	作为交接标准	50
5.5.3	作为自查标准	50
5.6	常见阻碍与解决方案	50
5.6.1	阻碍 1: ”我现在只是探索，没必要这么严格”	50
5.6.2	阻碍 2: ”满足 DoD 太花时间了”	51
5.6.3	阻碍 3: ”我们已经有实验管理工具了（如 MLflow、W&B）”	51
5.7	10 分钟动作：为当前最好的结果做 DoD 检查	52
6	实验记录自动化：缺的不是工具，是默认行为	55
6.1	故事引入：三个月后的”考古工作”	55
6.2	两层日志策略：机器精确 + 人类简洁	56
6.2.1	第一层：机器日志（run.json）	56
6.2.2	第二层：人类日志（run.md）	60
6.3	自动化工具：让记录成为零成本行为	61
6.3.1	在训练脚本中自动生成 run.json	61
6.3.2	用模板简化 run.md 编写	67

6.4	与现有工具集成	68
6.4.1	与 MLflow 集成	68
6.4.2	与 Weights & Biases 集成	69
6.5	日志查询与分析	70
6.5.1	查找最好的实验	70
6.5.2	对比两个实验的配置差异	71
6.6	常见问题与解决方案	73
6.6.1	Q1: run.json 太大了怎么办?	73
6.6.2	Q2: 忘了记录 run.md 怎么办?	73
6.6.3	Q3: 数据版本太大, 无法计算哈希怎么办?	74
6.7	10 分钟动作: 为下一个实验建立自动记录	74
7	AI 编码助手加入后, 工作流必须升级	76
7.1	故事引入: 当 AI 从提速器变成地雷制造机	76
7.2	AI 编码的三大陷阱	77
7.2.1	陷阱 1: 生成速度超过验证能力	77
7.2.2	陷阱 2: 重复生成导致不一致	78
7.2.3	陷阱 3: 过度依赖”顺手改一下”	78
7.3	升级后的工作流: 把 AI 当成”快速试错的初级程序员”	79
7.3.1	原则 1: 每次只做一个可验证的改动	79
7.3.2	原则 2: 每次改动附带”如何验证”	80

7.3.3	原则 3：禁止”顺手重构一大片”	81
7.3.4	原则 4：核心逻辑必须人工审查	82
7.3.5	原则 5：先补最小测试，再改核心逻辑	83
7.4	AI 规则页 (CLAUDE.md)：团队协作的共识	84
7.4.1	CLAUDE.md 模板	84
7.5	实战案例：AI 辅助的正确打开方式	89
7.5.1	案例 1：添加新模型组件	89
7.5.2	案例 2：重构数据加载	90
7.6	常见问题与解决方案	92
7.6.1	Q1：AI 生成的代码太复杂，看不懂怎么办？	92
7.6.2	Q2：AI 生成的代码有 bug，但改了又出现 新 bug	93
7.6.3	Q3：团队成员不遵守 AI 使用规范怎么办？	93
7.7	10 分钟动作：为下一次 AI 编码建立检查点	94
8	多路径探索如何不变垃圾堆	97
8.1	故事引入：从”灵活探索”到”不敢动任何东西” . .	97
8.2	为什么多路径探索容易失控	99
8.2.1	失控的三个阶段	100
8.2.2	根本原因：缺少”可丢弃”和”可合并”机制	101
8.3	核心机制：隔离 + 可丢弃 + 可对比	101
8.3.1	机制 1：每条路径必须隔离	101

- 8.3.2 机制 2: 明确的生命周期管理 103
 - 8.3.3 机制 3: 公平对比的基准线 108
- 8.4 每周清理仪式: 实验基地整理 111
 - 8.4.1 每周五下午的清理流程 (30 分钟) 111
 - 8.4.2 清理决策树 113
- 8.5 路径合并策略: 从探索到稳定主线 114
 - 8.5.1 合并前检查清单 114
 - 8.5.2 渐进式合并策略 115
- 8.6 常见问题与解决方案 116
 - 8.6.1 Q1: 路径太多了, 记不住怎么办? 116
 - 8.6.2 Q2: 不同路径的代码冲突怎么办? 117
 - 8.6.3 Q3: 删除路径后后悔了怎么办? 117
- 8.7 10 分钟动作: 整理当前的探索路径 118

9 让最后阶段不爆炸的 3 个提前动作 121

- 9.1 故事引入: 离 deadline 只剩一周的噩梦 121
- 9.2 为什么”最后阶段爆炸”几乎是必然 123
- 9.3 提前动作 1: 每周一次”可复现性自查”(15 分钟) 124
 - 9.3.1 为什么重要 124
 - 9.3.2 自查清单 (15 分钟完成) 124
 - 9.3.3 自查频率与时机 126
 - 9.3.4 常见陷阱 126

9.4 提前动作 2：每月一次” 债务盘点”（30 分钟） . . . 127

9.4.1 为什么重要 127

9.4.2 盘点清单（30 分钟完成） 127

9.4.3 债务可视化 130

9.5 提前动作 3：论文前三个月建立” 复现基线”（1 小时） 130

9.5.1 为什么重要 130

9.5.2 复现基线的内容 131

9.5.3 建立时机 135

9.6 紧急补救方案：如果已经到了最后阶段 135

9.6.1 如果距离 deadline 只剩 2 周 135

9.6.2 如果距离 deadline 只剩 1 周 137

9.6.3 事后补救 137

9.7 10 分钟动作：今天就做的自查 138

9.8 本章总结：预防胜于救火 139

10 带学生/合作者的版本：团队最小规范 141

10.1 故事引入：从” 各自为战” 到” 互相拖累” 141

10.2 为什么” 各自厉害” 不等于” 团队高效” 142

10.2.1 三大协作陷阱 143

10.2.2 根本原因：缺少” 团队规范” 144

10.3 最小规范 1：代码规范（从混乱到可读） 144

10.3.1 命名规范：让代码自解释	144
10.3.2 注释规范：写给未来的你和队友	146
10.3.3 PR 模板：即使一个人也要用	148
10.4 最小规范 2：实验规范（从口头到可查）	150
10.4.1 统一 run_id 命名	150
10.4.2 统一 config 管理	150
10.4.3 统一 logging 格式	152
10.5 最小规范 3：沟通规范（从口头到留痕）	153
10.5.1 周会规范：只看可复现的 run	153
10.5.2 异步沟通规范：用文档而非口头	155
10.5.3 Issue 追踪：问题和任务可查询	156
10.6 导师视角：如何带学生建立好习惯	158
10.6.1 第一天：设定期望	158
10.6.2 定期 Review：不仅看结果，更看过程	159
10.6.3 奖励好习惯	160
10.7 学生视角：如何在混乱的项目中生存	160
10.7.1 如果项目已经很混乱	160
10.7.2 如何请求帮助	162
10.8 冲突解决：常见团队问题	163
10.8.1 问题 1：代码风格不统一	163
10.8.2 问题 2：实验结果对不上	164

10.8.3 问题 3: 分工不明确导致重复劳动	164
10.8.4 问题 4: 导师和学生期望不一致	165
10.9 10 分钟动作: 建立团队第一个规范	166
10.10 本章总结: 规范不是束缚, 是高效协作的基础 . . .	169
A 模板库 (今天就能开始用)	170
A.1 PR/提交说明模板 (即使一个人也写)	170
A.2 实验记录 <code>run.md</code> (5 行就够)	170
A.3 Makefile 目标 (把复现变成习惯)	171
B AI 助手引发的真实研发“灾难”案例 (用于对照)	172
B.1 案例 A: 用 Copilot 开发整套应用, 最终推倒重写 .	172
B.2 案例 B: 让 Copilot 包办代码库, 反复返工且引入 安全/逻辑隐患	172
B.3 案例 C: 模型输出格式变动导致 workflow 崩溃	173
B.4 案例 D: 跨语言“翻译代码”看似完成, 实际大量 API 幻觉	173
B.5 案例 E: 把项目完整托付 ChatGPT, 20 天后拿不 到任何交付	173
B.6 案例 F: ChatGPT 代写 ML 模型“漂亮地犯错”, 险 些造成重大损失	174
B.7 案例 G: AI 协同编程引发版本混乱与代码“消失” .	174
B.8 案例 H: ChatGPT 输出不可再现, 科研复现性堪忧	174

前言

AI 时代把研究与开发的边界进一步抹平：从提出假设、搭建原型、跑通实验，到把结果固化为可复现的证据链，这一切都在更短的周期里完成；而 AI 编码助手让“写出能跑的代码”变得前所未有的容易，却也让“写出可信、可追溯、可复现的研究代码”变得更难。

我写这本《Research Engineering OS》，不是想再提供一套抽象的“方法论”，而是把我在学术机器学习/计算生物学研究中反复踩过的坑，压缩成一套可执行的默认行为：用规范减少返工、用模板降低协作成本、用检查清单在每天的节奏里提前消化最后阶段才爆炸的风险。

这本小书的核心观点很朴素：探索可以野，但产出必须能被清理；结论可以暂时脆弱，但证据链必须牢靠。你可以快速试错，但要为每一次“看起来有效”的结果留下足够的信息，使它在一周后、一个月后、换一台机器后依然能被复现、被质疑、被验证。

因此，本书会反复强调三件事：

- 实验是最小单位：记录的不是“改了哪些代码”，而是“这次实验由哪些版本与配置构成”。
- 默认自动留痕：让 `run_id`、`commit`、`config`、数据版本、环境摘要成为流水线的一部分。
- **DoD** 与清单前置：把论文阶段才会要求的严谨性，拆成日常可执行的小动作。

如果你正在做的工作属于“AI 时代的研究开发”(research development): 既要保持探索速度、又要对结果负责、还要与合作者高效沟通, 希望这本书能成为你桌面旁的一份最小操作系统。

Li Hongmin (李鸿敏)

Department of Computational Biology and Medical Sciences
Graduate School of Frontier Sciences, The University of Tokyo
5-1-5 Kashiwanoha, Kashiwa-shi, Chiba 277-8561, Japan

li-hongmin.github.io

lihongmin@edu.k.u-tokyo.ac.jp

1 你为什么总在最后推翻 (探索债 / 验证债 / 复现债)

1.1 故事引入：离截稿只剩几天，你突然不敢相信自己的结果

想象这样一个场景：离论文截稿只剩下最后几天，你好不容易把实验结果都跑完，正准备写论文结论。但这时你心里开始打鼓：某个关键实验好像还没验证另一种数据划分？某个 baseline 好像跑得不公平？你决定谨慎地重跑一下。

结果噩梦发生了：重新运行后指标跟之前不一样了，原本”显著领先”的方案突然不再明显；或者你发现换了随机种子结果就下降。顿时一身冷汗——几个月的成果仿佛建立在沙滩上。一边是临近的 DDL (deadline)，一边是站不住的结论，你不得不推翻重来。

这类”最后阶段的大爆炸”是不是很熟悉？

1.2 为什么会这样：三类债务在后期集中爆炸

为什么我们总是在”快要写论文/快要答辩/快要投稿”的最后关头，才发现结果站不住，于是不得不推翻重来？

这并非个例。我的观察是：这通常不是某一个单点 bug 引发的，

而是三类债务在前期被默许、在后期集中爆炸——探索债、验证债与复现债。

换句话说，我们在研究过程中为了图省事积累了不少”债”，一直拖到最后才被迫一起偿还。本书的目的，就是把这种临近 DDL 才火烧屁股式的返工，拆解成日常可管理的小步骤，避免悲剧重演。

1.3 症状表：你可能正在走向最后推翻

以下症状如果似曾相识，那么你很可能正在不知不觉地积累研究债务，为最终阶段的”爆炸”埋下伏笔：

- 结论需要”讲故事”才能自洽：你能解释为什么有效，却说不清在什么条件下会失效；你很少主动讨论负面结果与边界条件。
- 结果对环境/随机种子极其敏感：换一台机器、换一个驱动版本或者换一个随机种子，指标就漂移不定。
- 没有一条”干净主线”跑完全程：分支很多、脚本很多、输出很多，但你没法用 1 条命令从预处理跑到论文主指标。
- 对照实验不完整：baseline 不够强、评测口径不一致、关键 ablation 缺失，失败案例也没有解释。
- 图表依赖手工操作：表格和曲线靠复制粘贴/手动筛选 run；源数据一更新就要手工重做，因此越接近截止越不敢动。
- 你开始害怕重跑：你隐约知道只要重跑，结果可能对不上记忆，或者根本回不来。

中招越多，意味着你在透支未来：用现在的省事换取最后阶段的高压返工。

1.4 一个常见剧情：为什么”最后阶段爆炸”几乎必然发生

回顾很多项目的轨迹，会发现”最后阶段爆炸”几乎是一种必然结果。剧情往往是这样的：

1. 早期冲刺：为了尽快看到信号，你用最快但不一定规范的方式把代码拼起来：能直接改脚本就不新建模块，参数写死在代码里，实验输出散落在各处……总之先跑通再说。
2. 信号出现：你看到一个”看起来不错”的曲线，于是继续叠加各种 trick、调整数据划分、修改训练细节，力图把指标再抬一点；与此同时，你把对照实验与整理工作往后拖。
3. 论文压力来袭：你突然需要可复现的主结果、完整对照组、可解释的消融分析；你开始补这些验证。
4. 系统崩溃：一补就出事：结果不稳、baseline 追上来、隐藏 bug 或数据泄漏暴露；你发现自己无法回答那些”是不是偶然/是不是泄漏/是不是只对某个 seed 有效”的问题。

关键问题不在于”探索阶段不该快”——探索当然可以快，但快的前提是后续可清理、可追溯。否则你跑得越快，实际上是用更大的杠杆在积累债务，最后翻车的风险也越高。

1.5 解释模型：三种债务

为了更系统地解释上述问题，我们引入”债务”的概念：和软件工程的技术债类似，在科研工作中我们也会欠下不同形式的债——探索债、验证债与复现债。

1.5.1 探索债（Exploration Debt）

定义：为了更快试错，我们在代码与实验流程上暂时采取不规范的捷径；这类权宜之计累积的负担就是探索债。探索债本身并非恶魔，前提是它必须可清理、可丢弃、可回收。

典型信号：

- 充斥”临时脚本”：test.py、try1.ipynb、debug_old.py
……没人敢删。
- 输出目录混乱：outputs/ 下面堆满 final_final2/、exp_new_try/ 之类文件夹，关系不清。
- 同样逻辑被复制粘贴改了多份，代码库像拼凑起来的，一改就牵一发动全身。

真实成本：当你需要收敛主线时，根本抽不出干净路径：你分不清哪些是最终方案需要的、哪些是废案；你也很可能已经忘了某些结果怎么来的。

1.5.2 验证债（Validation Debt）

定义：为了尽快证明”有效”，我们跳过了本应有的对照实验与严格测试；这些被跳过的验证迟早要补，而且越临近发表代价越大。

典型信号：

- baseline 不够强，或与方法不共享评估口径（不可直接比较）。
- 缺少消融实验：你不知道到底是哪一个改动带来主要增益。
- 指标口径不一致：阈值/后处理/数据过滤在不同 run 里不一致。

真实成本：在质疑压力下站不住脚：审稿人往往要求的不是”更大模型”，而是”更严谨的对照与分析”。

1.5.3 复现债 (Reproducibility Debt)

定义：为了先跑出结果，我们没有及时固化环境、数据版本、超参配置与随机性；最终导致同一份代码未必跑得回同一个结论。

典型信号：

- 你记得某次 run 很好，但找不到当时的 commit/config/seed/数据版本。
- 依赖版本漂移：昨天能跑，今天装了某个包就坏了或性能大降。
- 训练脚本充满隐式默认值与硬编码：别人（包括未来的你）弄不清关键超参。

真实成本：复现债会在最关键时刻夺走主动权：你无法自信回答”结果是否稳定/是否可复现”，也无法从容应对论文复查与复现实验。

1.6 本章结论：默认行为，而非复杂工具

很多人遇到这些问题会寄希望于更复杂的工具链，但本章想强调：你需要的不是更复杂的工具，而是一套好的默认行为。

在 AI 时代，代码生成的门槛在降低，但验证与复现的成本并没有同步降低，甚至因为迭代更快而相对上升。

与其依赖繁重的事后补救，不如在日常研究中养成轻量默认习惯，把债务压在最低水平：

- 任何“看起来不错”的结果，都必须能被重新跑出来（至少一次）：先确认不是偶然波动。
- 任何影响结论的改动，都必须能被解释为可对照的实验差异：一变一测，控制变量。
- 任何探索路径都必须可丢弃：有价值的归档，无价值的清理，主线保持干净。

1.7 本书原则（贯穿后续章节的理念）

- 小步可验证：每次迭代只引入一个可解释变量变化，优先确保系统可跑、可测。
- 强制留痕：commit、config、seed、数据版本、环境摘要都视为实验产物的一部分。
- 稳定/探索隔离：稳定主线用于论文可复现，探索分支用于快速试错，通过清单验收连接。

1.8 你现在就能做的 10 分钟动作

如果你现在只做一件事：为当前最好的结果建立一个可复现的最小入口。

1. 集中输出：为最好的一次实验指定 `run_id`，把关键输出集中到对应目录。
2. 配置入档：新建一个最小 config 文件，把关键超参写进去。
3. 记录指纹：写下 commit、seed、数据版本与主要依赖版本（哪怕先手写）。

4. 独立重跑：在新终端/新进程里用该 config 重跑一次，确认结果回到同数量级。

如果顺利，你得到一个可复用的模板；如果不顺利，你提前暴露了问题——趁现在解决还来得及。

后续章节会把这 10 分钟动作扩展成一套可持续的 Research Engineering OS。

2 项目的最小单位不是”代码”，是”实验”

在科研项目中，如果你把最小单位理解成”代码”，自然会倾向于用 Git 来组织一切：提交 (commit)、分支 (branch)、合并 (merge)、回滚 (rollback) 等等。这种习惯在工程开发中非常有效，但在研究工作中往往失灵。原因在于：研究的最终产出并不是某段代码本身，而是一条可被验证的结论或发现。换言之，代码只是达到研究结论的手段之一，实验结果才是我们真正关心的成果。

结论来自实验。一个实验由一组可追溯的输入条件驱动，产生出可复查的输出结果。因此，本书将”实验”定义为项目操作的最小单位，并围绕它来组织目录结构、日志记录、自动化流程以及验收标准。通过将实验视作基本单元，我们可以确保每一次结果都有据可循，提升研究的可重复性和可靠性。

2.1 实验的定义：把”我改了点东西”变成可比的对象

简单来说，一个实验 = 代码版本 + 配置 + 数据版本 + 环境 + 输出 + 指标。用更直白的话描述就是：任何一次跑出来的结果，都必须能回答”它到底是怎么来的？”换言之，每当你”对代码或配置”改了点东西”并跑出结果时，这次运行都应当形成一个独立的、可比较的实验对象。

2.1.1 必须能回答的 6 个问题

为了保证实验结果的可追溯和可比较，每次实验至少要能回答以下六个关键问题：

1. 用的是什么代码？—指明代码版本，例如 Git 的 commit hash，以及当时代码库是否有未提交修改（dirty）。这确保我们知道使用了哪一版代码跑的实验。
2. 用的是什么配置？—指明所用的配置参数集合，比如配置文件路径，以及解析展开后的最终参数值。配置决定了实验的超参数和运行选项，必须明确记录。
3. 用的是什么数据？—指明所用的数据集版本或哈希，以及数据的划分方式（训练/验证/测试集划分）。数据不同会直接影响结果，因此需要精确描述数据来源和版本。理想情况下，能通过数据的哈希或清单文件 (manifest) 证明数据的一致性。
4. 用的是什么环境？—描述运行所依赖的环境信息，例如 Python 版本、依赖库版本、驱动版本，以及关键硬件配置（CPU/GPU 型号等）。环境差异可能影响实验结果的重现，如果有人在另一环境重跑，需要知道原始环境是什么。
5. 产出存在哪里？—清晰记录实验产出的各类文件的位置，比如模型权重、日志、预测结果、缓存的中间特征等等。产出物是后续分析和验证的基础，必须保存并能够找到。
6. 指标是什么，如何计算的？—给出实验评估使用的指标及其计算方法，包括评估脚本和任何后处理的细节。不同的指标计算口径会导致结果不可比，必须保证指标定义透明、一致。

上述六个要素缺一不可。如果有任何一个问题无法回答，那么这次实验就不是完全可复现的，得出的结论也会缺乏说服力。只有同时满足这六点，我们才能说”我们知道这次结果是怎么来的”，也才能在日后精确重现或比较不同实验。

案例：AI 生成”看似完整”的代码，但无法运行。 在跨语言迁移（”代码翻译”）的场景中，经常会出现 API 幻觉 的现象：模型生成的函数看起来很合理，但实际上根本不存在于目标语言的库中，导致生成的代码无法直接运行。最终研究人员不得不逐个文件进行重写修正（详见案例 B.4）。如果我们把项目的最小单位误当成了”代码”，可能会陷入只关注生成了多少文件、多少行代码的陷阱。然而，将”实验”作为最小单位的意义在于：我们验收的标准不是产出了多少文件，而是产出了可运行的成果，以及可对照评价的指标。换句话说，一次实验是否成功，不取决于改动了几行代码，而取决于你是否得到了可执行的模型或结果，以及明确的评测指标来证明改动的效果。

2.2 实验对象模型：将研究过程拆解成稳定的”五要素”

为了让研究流程中的各环节都清晰可控、易于组合，并方便工具自动化，我们可以用一组固定的概念对象来描述实验过程的要素。每个实验都涉及以下五种对象：

- **config（配置）**：一次运行的所有关键参数设置。它应当是可序列化的（便于保存记录），支持局部覆盖的（便于在默认配置上修改），并且可以方便地比较差异（diff）。config 定义了实验要怎么跑，比如模型架构参数、学习率、训练轮数等，是实验的蓝图。

- **dataset (数据集)**: 所使用的数据版本以及划分方式。数据集对象应该明确标识所用的数据，如通过版本号、数据哈希或清单文件来证明数据的一致性。这样可以确保不同实验使用相同的数据基础，或清楚地知道数据差异。
- **run (运行)**: 一次具体的代码执行过程。一个 run 通常会记录唯一的运行 ID (run_id)、时间戳、所用代码的 commit 版本、对应的 config、随机种子 (seed) 以及运行日志等信息。run 表示一次真实发生的实验过程，是把配置和代码付诸实践得到结果的过程记录。
- **artifact (产出物)**: 由一次 run 所生成的所有成果文件。例如训练得到的模型权重文件、模型对测试集的预测结果、缓存的中间特征、评估时生成的中间数据等等，都是 artifacts。artifact 是实验的直接输出，我们后续的分析、对比、报告都建立在这些产出物之上。保存 artifacts 能让我们随时检视或复用实验结果，而不必重复耗时的计算。
- **report (报告)**: 面向人类阅读的总结性内容，包括从实验中得出的图表、表格、关键结论句，以及对失败情况的分析等。report 可以看作将定量的实验结果转化为定性的洞见，往往汇总了多个 run 的指标和 artifact，提供给读者或决策者参考。它是实验成果的最终表达形式。

这些对象之间的关系可以用一句话来概括：

config + dataset + env (环境) 经过代码执行，生成一个 run；该 run 产出若干 artifact；基于 artifact 和指标计算结果，我们撰写出 report。

通过这种对象模型，我们将纷繁复杂的研究过程拆解成了几个稳定的”名词”对象，便于思考和沟通。例如，在讨论实验时，我们

可以明确地区分”这次实验用了哪个 config 和 dataset”、”产生了哪些 artifact”、”最终报告怎么写”等等。更重要的是，这种划分为后续的工具化打下基础——我们可以为每类对象制定规范（比如 config 用 YAML 文件管理，artifact 按照特定目录保存），实现实验流程的标准化和自动化。

2.3 run_id: 让每一次运行都可以被清晰指代

在撰写论文或报告时，你可能需要频繁引用某次特定设置下跑出的结果，例如：”我们用某个设置跑出来的最好结果是……”。如果这次实验没有一个稳定且清晰的名字，你在描述时就会非常含糊，不利于读者理解，也容易混淆不同实验。为避免这种情况，我们建议每次运行都生成一个唯一的 run_id，并尽量让这个 ID 可读且具备时间顺序。

一个实用的做法是将时间戳和简短描述结合，形成有序且易懂的命名。例如，可以按日期和启动时间命名，再加上对实验内容的简述：

2026-02-01_0930_baseline	(2026年02月01日09:30启动的 baseline)
2026-02-01_1130_ablation_noaug	(2026年02月01日11:30启动的 ablation_noaug)
2026-02-02_0045_sweep_lr3e-4	(2026年02月02日00:45启动的 sweep_lr3e-4)

这样的命名约定使得文件按字典序即是按时间顺序排列，一眼就能看出先后关系和大致内容。每个 run_id 都保证了唯一性和可读性，避免了”experiment1”、”experiment2”这类模糊且不可比较的名字。

在文件组织上，可以以 run_id 作为目录名，集中保存该次实验的所有输出。例如：

```
outputs/<run_id>/
```

run.json	# 关于此次运行的元信息（例如代码版本、启动时间等）
run.md	# 可选：记录对这次运行的描述、观察和初步结论
metrics.json	# 此次实验的各项指标结果
artifacts/	# 子文件夹，内含该 run 产出的模型、预测结果等

通过这种结构，我们能够方便地管理和查询实验结果。例如，当你想比较多个实验时，可以直接打开对应 run_id 目录下的 metrics.json 查看指标，或载入 artifacts 中的模型进行分析。

避免”final”混乱： 很多人在命名实验时喜欢用”final”（最终）字样，但经常出现的情况是做完”实验_final”之后又发现需要小改进，于是又有”实验_final_v2”乃至”final_final”。最后自己都搞不清哪个才是真正的最终结果，造成混乱和误解。这正是缺乏规范命名的典型问题。采用 run_id 的方案后，你不会再依赖这些含糊的称呼来标记最终成果，而是用清晰的时间和内容描述来标识每一次尝试。至于哪个实验是最终采纳的结果，可以在报告里明确指出使用哪个 run_id 即可。总之，不要再让”final”这样的字眼干扰你的实验管理；通过统一的运行命名，每次实验都有据可依，该第几次就是第几次，再也不会混淆。

2.4 一条实用原则：将”可比性”放在第一优先级

在研究中，决定进展快慢的往往不是模型训练所花的物理时间，而是由于实验之间缺乏可比性所导致的反复和疑惑。如果不同实验的条件不一致，那么哪怕结果数字出来了，你也很难判断差异源自何处，甚至可能因此推翻重做。常见的反面例子包括：

- 评价标准不一：基准方法 (baseline) 和你的新方法用了不同

的评估脚本，导致结果度量口径不一样，无法直接比较。这会迫使你花额外时间将两者重新放到同一标准下评估。

- 后处理方式不一致：实验 A 和实验 B 各自用了不同的结果后处理或过滤策略，导致评估指标不在同一尺度上。比如一个结果用了额外的阈值筛选而另一个没有，如果不统一处理，你很难说清哪个方法更好。
- 数据划分不一致：实验 B 临时换了数据集或划分方式但没有记录，结果拿出来和实验 A 比较时其实不公平——可能 B 用了更简单的测试集却声称方法更优。这种情况下，即使结果好了也毫无意义，因为比较不在同一基准上。

由于上述种种原因，我们应当始终牢记：任何重要的结论都必须来自同一条评估流水线，确保实验之间具有可比性。也就是说，在比较两个实验的结果时，除了有意改变的那个变量（例如模型结构的不同、超参数的不同等），其他所有环节——数据、评估标准、后处理方法、随机种子等——都应尽可能保持一致或有据可查。一旦发现有不一致的地方，要么在新的实验中纠正过来重新跑，要么明确地在报告中说明不同之处并避免直接比较。

以可比性为第一优先级，可能会让你在设计实验时多花一些心思去对齐各项条件，但是这样做反而加速了整体研究进度。因为你避免了日后由于不公平比较带来的反复试验和争论，一次实验的结论就能真正站得住脚、经得起推敲。

从本章开始，后续所有章节——包括仓库结构、Git 工作流、DoD（完成定义）、日志记录以及 AI 辅助工作流等内容——都将围绕实现“实验可追溯”和“结果可对照”这两大核心目标展开。本章奠定的理念将贯穿整个研究管理的方法论。

3 仓库结构就是你的第二大脑

研究代码的”混乱”常常不是因为能力不足，而是因为研究天然就是并行探索：同一时间你要维护多个假设、多个实现、多个实验入口、多个输出与多个图表。

当这些东西没有结构地堆在一起，你的大脑就会被迫充当索引：哪个脚本还能跑？哪个输出可信？哪个改动影响了指标？

仓库结构的目的是不是好看，而是降低认知负担：让你不用靠回忆就能判断”这段代码是否可删、这份输出是否可信、这条路径是否可复现”。

3.1 真实案例：快速堆代码的代价

在我早期使用 AI 编码助手时，曾经历过这样一次教训：为了快速验证一个想法，我让 Copilot 帮我生成了大量”能跑”的代码——数据加载、模型定义、训练循环、评估脚本等等，几个小时内就搭起了一个看似完整的框架。

初期的”成功”：代码确实跑通了，实验也出了结果。我兴奋地继续迭代，不断让 AI 帮我添加新功能：数据增强、不同的模型变体、各种评估指标……每次改动都”立竿见影”，代码量迅速

膨胀。

崩溃的开始： 两周后，当我需要准备论文的对照实验时，问题开始暴露：

- 我分不清哪个脚本是最新的，哪个是废弃的；
- 同样的数据加载逻辑被复制粘贴到了 5 个不同文件里，每个版本略有差异；
- baseline 和新方法用了不同的评估代码，结果根本不可比；
- 想要复现某个“很好的结果”，却找不到当时用的配置和数据版本。

推倒重来： 最终，我不得不停下所有新实验，花了整整三天时间重写几乎所有代码。这次重写不是因为 AI 生成的代码有 bug，而是因为缺乏结构：可复用的核心逻辑和一次性实验脚本混在一起，快速试错的代码没有及时清理，输出散落各处难以追溯。

这次经历让我深刻理解：AI 可以帮你快速产出代码，但结构必须由人来设计。如果在最开始就把“稳定”和“探索”分离开，把输出按 `run_id` 组织，后续就不会陷入这种混乱。

案例参考： 这类问题并非个例。当你用 AI 快速堆出一个“能跑”的仓库，却没有把可复用代码与一次性实验入口隔离开来，最终常见结局是：每个模块都要重写，AI 生成片段几乎全被替换（见案例 [B.1](#)、[B.2](#)）。结构是避免这种返工的第一道防线。

3.2 一个可直接照抄的目录结构（研究友好）

```
repo/
  src/                # 核心库：可复用、可测试、可维护（慢变量）
  experiments/        # 实验入口：一次性 glue 代码（快变量，可删）
  configs/            # 统一配置：yaml/json（可diff，可追溯）
  data/               # 只放指针或小样本；大数据用外部管理
  outputs/            # 运行产物：按 run_id 组织（可清理/可归档）
  reports/            # 论文图表与结论：从 outputs 自动生成
  scripts/            # 数据准备/下载/评估等工具脚本
  tests/              # 单测 + smoke test（守住底线）
  Makefile            # 常用入口：train/eval/reproduce/
test
  README.md
  CLAUDE.md           # AI 编码规则（建议）
```

3.3 快变量与慢变量：把”稳定”从”探索”里分离出来

建议把仓库里的东西分成两类：

- **慢变量 (stable)**：会被长期维护、反复复用、需要测试覆盖的部分。
- **快变量 (explore)**：为某个假设快速试错的入口脚本、短命 glue、一次性分析。

在本书的术语里：`src/` 是慢变量，`experiments/` 是快变量。

经验法则：探索可以脏，但核心库必须干净；探索可以快，但评估必须稳定。

为什么这个分离如此重要？在我重写代码的经历中，最大的困扰就是无法区分资产和消耗品。当所有代码都混在一起时，你不敢删除任何东西（怕删错重要功能），也不敢大改（怕影响到其他实验）。一旦明确了 `src/` 是资产、`experiments/` 是消耗品，心理负担就会大大减轻：

- 对 `src/` 的改动要谨慎，需要测试；
- 对 `experiments/` 的改动可以随意，试完就删。

3.4 每个目录的 Definition of Done (DoD)

目录名只有在“什么该放、什么不该放”足够清晰时，才会真正降低混乱。

3.4.1 `src/`：核心库（可复用、可测试）

- 放可复用模块：数据加载、模型组件、损失、评估、通用工具。
- 必须可测试：至少有 smoke test 覆盖关键链路。
- 禁止硬编码：不要出现只对某个 run 有用的路径/参数。

反例：在我的重写案例中，原本的“数据加载”代码硬编码了某个特定实验的路径和预处理方式，导致新实验不得不复制粘贴并修改。如果当初就把路径和参数作为函数参数传入，就不会有这个问题。

3.4.2 `experiments/`：实验入口（可丢弃）

- 只放入口与 glue：允许短命，用完即删。

- 任何被证明有价值、会复用的逻辑，一旦稳定就迁移到 `src/`。

实践建议： 每个实验脚本以日期或 `run_id` 命名, 比如 `2026-02-01_baseline`。这样你一眼就能看出哪些是老实验，哪些是新实验。定期（比如每周）清理超过一个月且没有价值的脚本。

3.4.3 `configs/`: 配置（可追溯）

- 每个”论文候选结论”都必须对应一个 `config`（或其可追溯的生成方式）。
- `config` 必须能展开成最终参数（避免默认值漂移）。

3.4.4 `outputs/`: 产物（可清理/可归档）

- 只放运行产物，并且按 `run_id` 组织。
- 禁止覆盖：同一个 `run_id` 目录不复用/不手工改。
- 重要产物做归档：进入长期存储，仓库不背大二进制。

3.4.5 `reports/`: 图表与结论（可再生成）

- 尽量只放脚本生成的图表/表格与结论草稿。
- 论文图表必须能从 `outputs/` 重新生成，避免手工拖拽。

3.4.6 `tests/`: 测试（守住底线）

- 至少一个 1-3 分钟的 `smoke test`: 跑通数据加载 → 前向 → 损失 → 评估。

- 关键函数做断言：shape、NaN、范围、数据泄漏信号等。

3.5 入口收敛：让”怎么跑”变得显眼

研究里最常见的浪费是：别人（包括未来的你）不知道怎么跑。

建议把所有常用入口收敛到 Makefile（或等价任务工具）：

- `make test`
- `make train CONFIG=...`
- `make eval RUN=...`
- `make reproduce RUN=...`

当入口足够少、足够稳定时，你就能把复杂性关在内部，把可复现性暴露在外部。

3.6 渐进式重构：从混乱项目迁移到良好结构

如果你已经有一个”混乱”的老项目，不要试图一次性推倒重来。

以下是一个循序渐进的重构流程：

3.6.1 第一步：识别并区分慢变量与快变量

1. 审视现有代码：扫描整个项目，标记出哪些模块是核心功能（会长期维护、反复使用），哪些是一次性试验脚本。
2. 新建目录：
 - 创建 `src/`，把确定会复用的模块迁移进去；

- 创建 `experiments/`，将各种杂散的运行脚本和临时代码移入其中。
3. 完成粗分层：这一步完成后，项目结构会开始变清晰，你脑中的索引负担也随之减轻。不要求完美，先把明显的部分分开即可。

3.6.2 第二步：提取配置与参数

1. 找出硬编码：扫描代码，找出所有硬编码的关键参数（学习率、batch size、路径等）。
2. 创建配置文件：在 `configs/` 目录下创建 YAML 或 JSON 文件，把参数集中管理。
3. 逐步替换：不要一次性改完所有代码，而是每次重构一个脚本，确保它能正常运行后再继续下一个。

实践经验： 在我的重构过程中，光是提取配置这一步就帮我发现了 3 个隐藏的 bug——原来不同脚本里“看似相同”的参数其实值不一样，导致结果不可比。

3.6.3 第三步：统一输出路径

1. 设定规范：决定使用 `outputs/<run_id>/` 的结构。
2. 修改代码：调整所有实验入口，让输出都按照 `run_id` 归档。
3. 清理旧输出：将散落的旧输出文件整理归档或删除，保持 `outputs` 目录干净。

3.6.4 第四步：添加基本测试

1. 编写 **smoke test**：创建一个 1-3 分钟的快速测试，验证核心流程能跑通。
2. 每次重构后运行：确保改动没有破坏基本功能。
3. 逐步增加覆盖：随着重构进展，逐步为关键函数添加单元测试。

关键原则： 循序渐进很重要。不要试图一次性做完全部重构，而是每步都确保现有功能正常再进行下一步。我的重写用了三天，但如果当初采用渐进式重构，可能一周内分散进行，不会影响正常实验进度。

3.7 多任务/多项目的目录层级管理

当面对多个相关但独立的研究任务时，如何组织目录是一个重要问题。

3.7.1 原则：优先考虑独立仓库

最佳实践：每个研究项目使用独立的代码仓库。这样可以：

- 避免不同项目的依赖冲突；
- 确保版本控制的独立性；
- 简化复现流程（每个项目有独立的环境和依赖）。

经验法则： 如果两个项目的依赖版本、数据集、运行环境差异较大，强烈建议分仓库。

3.7.2 单仓库多项目的分层结构

如果确实需要在一个仓库中管理多个相关任务（比如同一论文的多个实验、或者共享大量代码的子项目），可以采用以下结构：

```
repo/
  projectA/
    src/
    experiments/
    configs/
    outputs/
    README.md
  projectB/
    src/
    experiments/
    configs/
    outputs/
    README.md
  common/
    src/                # 跨项目共享的核心代码
    tests/              # 共享功能的测试
    scripts/            # 通用工具脚本
  README.md             # 总体说明
  Makefile              # 跨项目的通用命令
```

3.7.3 命名约定和环境隔离

- 配置文件命名：使用项目前缀，如 `configs/projectA_baseline.yaml`、`configs/projectB_ablation.yaml`。
- 输出目录：可以在项目子目录下各自维护 `outputs`，或者统

一在根目录但使用项目名作为前缀：`outputs/projectA/<run_id>/`。

- 环境管理：即使代码在一个仓库，也建议为不同项目维护独立的虚拟环境或 Docker 容器，防止依赖冲突。

3.7.4 何时共享代码，何时复制代码

共享代码到 `common/` 的时机：

- 多个项目都需要使用的基础工具函数；
- 通用的数据加载或预处理逻辑；
- 标准化的评估指标计算。

复制代码的时机：

- 代码还在快速变化，不同项目需求可能分化；
- 共享会导致过度耦合，影响独立性；
- 项目即将结束，未来不再需要同步更新。

实践建议：开始时宁可复制，等到代码真正稳定、确认需要共享时再提取到 `common/`。过早抽象会导致频繁修改共享代码，反而增加维护负担。

3.8 10 分钟动作：把当前项目”分层一次”

如果你现在只做一件事：把当前仓库粗略分成慢变量与快变量。

1. 新建 `src/`，把确定会复用的模块迁进去。
2. 新建 `experiments/`，把所有入口脚本搬进去，允许它们短命。
3. 新建 `configs/`，把关键参数从脚本里抽出来。
4. 规定输出统一进 `outputs/<run_id>/`。

你会立刻感觉”可控性”提升：因为你开始能区分什么是资产、什么是一次性消耗品。

从个人经验看： 如果我当初在开始使用 AI 编码助手时就建立了这个结构，那三天的重写工作完全可以避免。良好的结构不是为了好看，而是为了让你的大脑不再被迫记住一切细节，让仓库本身成为你可靠的”第二大脑”。

4 Git 不是用来“保存代码”， 是用来“证明历史”

4.1 故事引入：审稿意见要求复现，你却找不回 当时的代码

论文提交三个月后，审稿意见回来了。其中一条写得很直接：“请提供代码和数据，我们想要复现表 3 的结果。”

你心里一紧——赶紧打开代码仓库。但眼前的景象让你后背发凉：

- Git 历史只有寥寥几次提交：“initial commit”、“update”、“fix bug”、“final version”；
- 论文里的结果是三个月前跑的，你已经记不清用的是哪个版本的代码；
- 代码目录里有多个版本的实验脚本：`train.py`、`train_v2.py`、`train_final.py`，你不确定用的是哪个；
- 更糟的是，你发现最近为了新实验已经大改了模型代码，现在的版本跑不出论文里的数字。

你只能硬着头皮回复：“我们正在整理代码，会尽快提供。”然后开始痛苦的“考古工作”——试图从记忆、聊天记录、实验笔记里拼凑出当时的代码状态。

这个场景你熟悉吗？

4.2 为什么”随手 commit”救不了你

很多人以为自己在用 Git，但实际上只是把 Git 当成了”云盘”：

- 改了一堆代码，最后一起 commit，message 随便写个”update”；
- 从不用分支，所有改动都在 main 上叠加；
- 实验跑完了才想起来 commit，这时候代码已经又改了；
- commit 历史里找不到”哪个版本对应哪个实验结果”。

这种用法的问题在于：你失去了 Git 最核心的价值——作为”历史证明工具”的能力。

在工程开发中，Git 的主要作用是协作和回滚。但在研究中，Git 的核心价值是证明：

- 证明这个结果是用哪个版本的代码跑出来的；
- 证明论文里的每个实验都有对应的代码版本；
- 证明你可以回到任意历史版本，重新跑出相同的结果。

4.3 研究中的 Git 使用陷阱

4.3.1 陷阱 1：commit 粒度太大，找不到关键改动

症状：一个 commit 包含了十几个文件的改动，涉及数据处理、模型结构、训练流程等多个方面。commit message 只写了”improve

model”。

后果：

- 无法定位某个指标变化是由哪个改动引起的；
- 想要回滚某个错误改动，却发现无法单独撤销；
- 几个月后再看，完全想不起来这个 commit 做了什么。

正确做法：

- 每个 commit 只包含一个逻辑改动；
- commit message 写清楚”改了什么”和”为什么改”；
- 遵循”原子性原则”：每个 commit 都应该让代码保持可运行状态。

4.3.2 陷阱 2：实验和代码改动时间错位

症状：先改代码跑实验，结果不错，过了两天才 commit；或者 commit 之后又临时改了几个参数重跑。

后果：

- 产生了结果的代码版本（commit）其实对不上；
- 别人（包括未来的你）拿着 commit hash 复现，结果对不上；
- 审稿人要求复现时，你根本找不到准确的代码版本。

正确做法：

- 先 commit，再跑实验；

- 每次实验的 run.json 记录当时的 commit hash 和 dirty 状态；
- 如果临时改了代码，要么重新 commit，要么在 run 记录里注明 dirty 修改内容。

4.3.3 陷阱 3：分支使用不当，主线混乱

症状：所有实验都在 main 分支上做，各种尝试性改动和稳定代码混在一起；或者创建了很多分支但从不清理，分支之间关系混乱。

后果：

- main 分支不稳定，充斥着各种试验性代码；
- 想要找到”论文复现版本”时，不知道该用哪个分支；
- 分支太多导致团队成员不知道该基于哪个分支开展新工作。

4.4 适合研究的 Git 分支策略

与工程项目不同，研究项目的分支策略需要平衡两个需求：

- 稳定性：论文结果必须有一个干净、稳定的代码版本支撑；
- 探索性：新想法需要快速试错，不能被流程束缚。

4.4.1 推荐的分支结构

main (或 stable)：

- 只接受验证通过的改动
- 每个 merge 必须经过 DoD 检查（见第5章）

- 确保任何时候都可以复现论文结果

`exp/<hypothesis-name>:`

- 每个实验假设一条分支
- 命名要清晰: `exp/attention-ablation`, `exp/data-augmentation`
- 短命分支: 验证完就合并或删除
- 允许"脏"的快速迭代

`archive/<paper-version>:`

- 论文提交、发表等重要节点的存档分支
- 从 `main` 分支创建, 不再合并回去
- 永久保留, 确保可追溯

4.4.2 典型工作流程

场景 1: 验证新假设

1. 从 `main` 创建新分支: `git checkout -b exp/new-loss-function`
2. 在分支上快速迭代、试错, `commit` 可以随意
3. 跑出有希望的结果后, 整理代码
4. 创建规范的实验记录 (`config + run.json`)
5. 合并回 `main`: `git checkout main && git merge exp/new-loss-function`
6. 删除实验分支: `git branch -d exp/new-loss-function`

场景 2: 论文提交

1. 确保 main 分支所有论文实验都可复现
2. 创建存档分支: `git checkout -b archive/icml2026-v1`
3. 在 main 上打 tag: `git tag -a paper-icml2026-v1 -m "ICML 2026 submission version"`
4. 推送 tag: `git push origin paper-icml2026-v1`

场景 3：并行探索多个方向

1. 同时创建多个实验分支：
 - `exp/architecture-search`
 - `exp/data-augmentation`
 - `exp/loss-function`
2. 各分支独立推进，互不干扰
3. 每个分支的实验产物用独立的 `run_id` 管理
4. 有价值的改动逐个合并回 main
5. 无价值的分支直接删除

4.5 用 Tag 标记里程碑：让论文结果永久可追溯

Tag 是 Git 中被严重低估的功能。对于研究项目，tag 的价值在于：

- 为论文的每个关键版本打上永久标记；
- 即使 main 分支继续演进，也能精确回到历史版本；
- 提供清晰的版本命名，便于引用和复现。

4.5.1 推荐的 Tag 命名规范

论文版本

paper-<venue>-<version>

例如: paper-icml2026-v1, paper-icml2026-revision

实验组

exp-<experiment-name>

例如: exp-ablation-study, exp-baseline-comparison

主要结果

result-<result-name>

例如: result-table3-main, result-fig2-comparison

里程碑

milestone-<description>

例如: milestone-first-sota, milestone-reproducible-baseline

4.5.2 Tag 使用实践

为论文的每个重要实验打 tag:

跑完主实验后立即打 tag

```
git tag -a result-main-experiment -m \
```

```
"Main results reported in Table 2, config: configs/main.yaml"
```

记录关键信息在 tag message 里

```
git tag -a result-ablation-study -m \
```

```
"Ablation study results (Table 3)
```

```
Run IDs: 2026-02-01_1030_ablation_*
```

```
Config: configs/ablation_*.yaml
```

Key finding: attention mechanism contributes 5% improvement"

复现时直接切换到 **tag**:

```
# 查看所有实验相关的 tag
```

```
git tag -l "result-*
```

```
# 切换到特定实验版本
```

```
git checkout result-main-experiment
```

```
# 复现实验
```

```
make reproduce CONFIG=configs/main.yaml
```

4.6 实验产物不进 Git: 用.gitignore 保持仓库干净

核心原则: Git 管理源代码和配置, 不管理实验产物。

4.6.1 什么不应该提交到 Git

- 模型权重: 通常很大 (几百 MB 到几 GB), 用专门的模型管理工具 (如 DVC、Git LFS、或云存储)。
- 训练日志: outputs/ 下的所有运行产物, 按 run_id 组织后归档或清理。
- 中间数据: 缓存的特征、预处理结果等, 应该可重新生成。
- 数据集: 原始数据通常由外部管理, 只在 data/ 里放小样本或数据指针 (清单、下载脚本)。
- 虚拟环境: venv/、.conda/ 等目录, 用 requirements.txt 或 environment.yaml 代替。

4.6.2 推荐的.gitignore 模板

```
# Python
__pycache__/*
*.py[cod]
*$py.class
*.so
.Python

# 虚拟环境
venv/
env/
.conda/

# 实验产物
outputs/
runs/
checkpoints/
*.pt
*.pth
*.ckpt
*.h5

# 数据（除非是小样本）
data/raw/
data/processed/
*.csv
*.parquet

# 日志
*.log
```

```
logs/
wandb/

# 临时文件
.DS_Store
*.swp
*.swp
*~

# IDE
.vscode/
.idea/
*.iml

# 例外：保留小样本数据和配置
!data/samples/
!configs/
```

4.7 常见问题与解决方案

4.7.1 Q1: 代码已经改了很多，怎么补救？

如果你的仓库历史已经很混乱，不要试图”重写历史”（除非你非常熟悉 Git rebase）。推荐的做法：

1. 设定基准点：在当前状态打一个 tag: `git tag baseline-before-cleanup`
2. 从现在开始规范：
 - 每个新实验使用独立分支

- 每个 commit 保持原子性
- 重要结果立即打 tag

3. 历史问题逐步修复：

- 识别出论文关键实验对应的代码版本，补打 tag
- 在 README 或文档里记录”历史版本对应关系”
- 新实验都用规范流程，旧实验尽力追溯

4.7.2 Q2：团队协作时如何统一分支策略？

- 写进 **README**：把分支命名规范、tag 使用方法写进文档。
- 设定保护规则：在 GitHub/GitLab 上设置 main 分支保护，禁止直接 push，必须通过 PR/MR。
- **Code Review**：合并到 main 前，检查是否满足 DoD（第 5 章），是否有完整的实验记录。
- 定期清理：每周一次会议，集体清理无用的实验分支，归档重要的 tag。

4.7.3 Q3：如何处理”dirty”状态的实验？

有时你临时修改了代码跑实验，还没来得及 commit，这就是”dirty”状态。

记录策略：

- 在 run.json 里记录 "git_dirty": true
- 同时记录 diff: `git diff > outputs/<run_id>/changes.patch`
- 在 run.md 里注明临时改动的内容和原因

事后补救：

- 如果结果有价值，立即把改动 commit 并打 tag
- 如果只是临时试验，在 run.md 里记录即可，不必 commit

4.8 实战案例：从混乱到清晰的 Git 历史

4.8.1 重构前（反面教材）

```
* a3f2d1c (HEAD -> main) update
* f8d9e0a fix
* 1b2c3d4 add new feature
* 9e8d7f6 initial commit
```

无法从历史看出任何有用信息，所有论文实验找不到对应版本。

4.8.2 重构后（正面示例）

```
* d1e2f3g (tag: paper-icml2026-v1, main) Merge exp/final-ablation
|           Paper results ready for submission
|\
| * c4d5e6f (exp/final-ablation) Add ablation study for attention
| * b3c4d5e Configure ablation experiments
|/
* a1b2c3d (tag: result-main-experiment) Main experiment: achieved
    Run ID: 2026-02-01_1030_main_run
    Config: configs/main_experiment.yaml
* 9a8b7c6 (tag: milestone-baseline) Establish reproducible baseline
    All baseline experiments validated
```

```
* 8f7e6d5 Fix data preprocessing bug in train/val split
* 7e6d5c4 Add comprehensive smoke test
* 6d5c4b3 Refactor data loading module
```

清晰的历史，每个重要节点都有标记，可以随时回溯。

4.9 10 分钟动作：为当前项目建立 Git 基线

如果你现在只做一件事：为你的项目建立一个清晰的 Git 基线。

1. 检查当前状态：

```
git status
git log --oneline -10
```

2. 如果有未提交改动，决定处理方式：

- 有价值的改动：整理后 commit，写清楚 message
- 临时试验：记录到 run.md，然后 git stash
- 无用改动：git checkout . 撤销

3. 为当前稳定版本打基线 tag：

```
git tag -a baseline-$(date +%Y%m%d) -m \
    "Baseline before implementing git workflow"
```

4. 设置规范的.gitignore：

```
# 使用前面提供的模板
curl -o .gitignore <模板链接>
```

```
# 或手动创建
```

```
git add .gitignore
```

```
git commit -m "Add comprehensive .gitignore for research"
```

5. 写下分支命名规范：在 README.md 里添加一节”Git Workflow”，记录：

- main 分支用途
- exp/ 分支命名规范
- tag 使用方法

从现在开始，每个新实验都按规范使用分支和 tag，让 Git 真正成为你的”历史证明工具”。

5 从”能跑”到”可信”只差一个 Definition of Done

5.1 故事引入：那些”差不多就行”的代价

凌晨两点，你终于跑出了一个”看起来不错”的结果——测试准确率 94.3%，比 baseline 高了 3 个百分点。你兴奋地截了图，发到团队群里：”新方法有效！”

三天后，当你准备写论文时，想要重新跑一遍验证结果。你打开代码，犹豫了：

- 用的是哪个配置文件？有三个类似的 yaml，记不清了。
- 数据是用的哪个版本？好像临时改过一次划分方式。
- 随机种子是多少？忘了记录。
- 当时的代码是 commit 了吗？还是有临时修改？

你硬着头皮重跑，结果出来了：92.7%。比之前低了 1.6 个百分点。

心里一沉——到底哪次是对的？还是两次都不可靠？

这个场景的问题在于：你没有明确的标准来判断”一个实验是否完成”。

在软件工程中，有一个概念叫 **Definition of Done (DoD)**——完成

的定义。它回答的问题是：”什么时候我可以说这个任务真的完成了？”

在研究中，我们同样需要 DoD，但标准不同：

- 工程的 DoD：代码能跑、测试通过、文档齐全。
- 研究的 DoD：结果可信、可复现、可对比。

5.2 为什么”能跑”不等于”完成”

在研究过程中，有太多”看似完成，实则埋雷”的情况：

5.2.1 埋雷 1：”跑出结果”但无法复现

症状：你看到了一个好结果，但缺少完整的环境记录、配置信息、数据版本。几天后想重跑，结果对不上。

真实成本：

- 审稿人要求复现时你手足无措；
- 团队成员无法基于你的结果继续工作；
- 最坏情况：论文因为”结果不可复现”被拒。

5.2.2 埋雷 2：”改进有效”但不知道为什么

症状：你同时改了三个地方，结果确实变好了，但你不知道是哪个改动起作用，也没有做消融实验。

真实成本：

- 审稿人要求解释机制时你答不上来；
- 后续改进不知道该保留哪些、舍弃哪些；
- 可能误把无效甚至有害的改动当成关键贡献。

5.2.3 埋雷 3: ”对比实验”但评估口径不一致

症状：你的方法和 baseline 用了不同的评估脚本，或者后处理方式不同，看起来你的方法更好，但实际上比较不公平。

真实成本：

- 审稿人指出评估不公平，要求重做；
- 重新跑后发现优势不复存在；
- 浪费大量时间在”公平化”评估上。

5.3 论文候选结论的 DoD 清单

以下清单适用于任何”可能写进论文”的实验结果。建议原样贴到项目 README 里，作为团队共识。

5.3.1 最小 DoD (5 项必须满足)

1. 一条命令可复现主指标

例如：

```
make reproduce RUN=2026-02-01_1030_main_experiment
```

或：

```
python reproduce.py --run_id=2026-02-01_1030_main_experi
```

必须能从零开始（给定环境和数据），用一条命令复现出论文报告的指标（允许小幅波动）。

2. run 记录齐全

每个重要实验必须有完整的 run 记录，至少包括：

- Git commit hash（代码版本）
- Config 文件路径及内容（所有超参数）
- Random seed（随机数种子）
- 数据版本（数据集版本号、哈希或清单文件）
- 环境摘要（Python 版本、关键库版本、GPU 型号等）

推荐格式见第 6 章的 `run.json` 模板。

3. baseline 与 ablation 使用同一评估脚本

所有对比实验必须：

- 使用完全相同的评估代码；
- 使用完全相同的数据划分；
- 使用完全相同的后处理方式；
- 指标计算逻辑一致（比如同样的阈值、同样的平均方式）。

验收标准：能指出一个统一的评估脚本，所有方法的指标都来自这个脚本。

4. 至少一个 smoke test 能在 1-3 分钟跑完

Smoke test（冒烟测试）是快速验证核心流程的测试，目的是尽早发现明显错误。

必须覆盖的关键环节：

- 数据加载（能正确读取并返回正确形状的数据）

- 模型前向传播（不崩溃、输出形状正确）
- 损失计算（数值正常、无 NaN）
- 评估流程（指标计算正确）

实现建议：用一个极小的数据子集（比如 10 个样本），跑 2-3 个迭代，确保流程完整。

5. 结果图表由脚本生成，不手工拖拽

论文中的所有图表必须能从 outputs/ 的原始数据自动生成。

禁止的做法：

- 手工从日志里复制数字粘贴到 Excel；
- 手工调整图表样式后截图；
- 图表的源数据文件找不到。

推荐做法：

- 在 reports/ 目录下放置生成脚本(如 plot_main_results.py)；
- 脚本读取 outputs/<run_id>/metrics.json，生成图表；
- 图表保存为可编辑格式（如 PDF）和源数据（如 CSV）；
- 在 Makefile 里加入 make plots，一键生成所有图表。

5.3.2 增强 DoD（推荐额外满足）

满足最小 DoD 后，以下几项可以进一步提升结果可信度：

1. 多次运行的统计结果

对于随机性较大的实验，单次运行不足以证明有效性。推荐：

- 至少用 3-5 个不同随机种子运行；

- 报告均值和标准差（或置信区间）；
- 在 run 记录里列出所有 seed 对应的 run_id。

2. 失败案例分析

诚实记录方法的局限性：

- 哪些情况下方法效果不好？
- 有没有明显的失败样本？
- 对超参数的敏感程度如何？

这不仅增强可信度，也为后续改进指明方向。

3. 完整的消融实验

对于包含多个改进的方法，必须有消融实验（ablation study）

回答：

- 每个改进单独贡献了多少？
- 哪些改进是关键的，哪些是边际的？
- 改进之间有没有交互作用？

4. 代码质量检查

使用自动化工具检查代码质量：

- Linter（如 flake8, pylint）：代码风格检查
- Type checker（如 mypy）：类型检查
- 单元测试覆盖率：关键函数必须有测试

在 CI 中自动运行这些检查。

5. 数据泄漏检查

确保训练集和测试集严格隔离：

- 数据划分前打印样本数，检查总数是否一致；
- 检查训练集和测试集有无交集（用样本 ID 或哈希）；
- 时间序列数据：确保测试集时间晚于训练集。

5.4 DoD 检查清单：从”能跑”到”可信”的操作步骤

5.4.1 实验完成后立即做（5 分钟）

1. 记录 **run** 信息

```
# 自动生成 run.json（见第6章工具）  
python log_run.py --run_id <id>
```

2. 手写 **run.md** 关键信息（不超过 5 行）

- 这次实验的假设是什么？
- 主要改动是什么？
- 结果如何（用一句话概括）？
- 下一步打算做什么？
- 有什么值得注意的风险或异常？

5.4.2 准备写论文前做（30 分钟）

1. 复现验证

在新终端（或新环境）里运行复现命令：

```
make reproduce RUN=<论文候选结果的 run_id>
```

检查：

- 是否能顺利运行（不报错）？
- 结果是否在合理范围内（相差不超过 1-2% 或一个标准差）？

- 如果差异较大，排查原因（环境、数据、随机性）。

2. 对比实验检查

检查所有要对比的方法：

```
ls outputs/  
# 找到所有 baseline 和 ablation 的 run_id
```

确认：

- 是否使用了同一份评估脚本？（可以通过 run.json 的脚本路径或哈希确认）
- 数据划分是否一致？
- 评估参数是否一致？

如果发现不一致，必须重跑其中某些实验，统一评估口径。

3. 运行 smoke test

```
make test  
# 或：  
pytest tests/test_smoke.py
```

确保核心流程没有被后续改动破坏。

4. 生成图表

```
make plots
```

检查生成的图表：

- 是否反映了最新的实验结果？
- 数值是否与 run 记录一致？
- 坐标轴、图例是否清晰？

5.4.3 论文提交前做（1 小时）

1. 完整性自查

对照 DoD 清单，逐项确认：

- ☐ 所有论文引用的实验都有 run_id
- ☐ 所有 run_id 都有完整的 run.json
- ☐ 所有对比实验使用同一评估脚本
- ☐ Smoke test 通过
- ☐ 所有图表可脚本生成

2. 为关键实验打 Git tag

主实验

```
git tag -a result-main-table2 -m \  
"Main results in Table 2, run_id: 2026-02-01_1030_main"
```

消融实验

```
git tag -a result-ablation-table3 -m \  
"Ablation study in Table 3, run_ids: 2026-02-01_14*"
```

推送 tag

```
git push origin --tags
```

3. 编写复现文档

在 README 或单独的 REPRODUCE.md 里写清楚：

- 环境安装步骤（一条命令或脚本）
- 数据准备步骤（下载、预处理）
- 复现每个表格/图表的命令

- 预期的运行时间和资源需求

示例：

```
# 复现说明

## 环境安装
conda env create -f environment.yaml
conda activate research-env

## 数据准备
bash scripts/download_data.sh
python scripts/preprocess.py

## 复现主实验 (Table 2)
make reproduce RUN=2026-02-01_1030_main
# 预期时间：2 小时（单卡 V100）
# 预期指标：accuracy 94.3% ± 0.5%

## 复现消融实验 (Table 3)
bash scripts/reproduce_ablation.sh
# 预期时间：6 小时（单卡 V100）
```

5.5 团队如何使用 DoD

5.5.1 作为合并标准

在团队协作中，DoD 可以作为代码合并到主分支的门槛：

1. 实验分支想要合并到 main，必须满足最小 DoD；

2. Code review 时，reviewer 按照 DoD 清单检查；
3. 不满足 DoD 的代码不能合并，必须补充完整。

5.5.2 作为交接标准

当项目需要交接（比如学生毕业、成员离职）时，DoD 确保知识不流失：

- 所有重要实验都有完整记录，新成员可以复现；
- 代码质量有保证（有测试、有文档）；
- 数据和模型有清晰的存储和访问说明。

5.5.3 作为自查标准

即使是个人项目，DoD 也能帮你避免”自己骗自己”：

- 定期（比如每周）检查未完成 DoD 的实验，补充记录；
- 写论文前批量检查，避免临时抱佛脚；
- 养成习惯后，DoD 会成为自然的工作流程，不再是额外负担。

5.6 常见阻碍与解决方案

5.6.1 阻碍 1：”我现在只是探索，没必要这么严格”

反驳：探索阶段可以降低 **DoD** 标准，但不能没有标准。

推荐的”探索阶段简化 DoD”：

- 不要求多次运行统计；
- 不要求完整的消融实验；
- 但必须：记录 commit、config、seed，确保能重跑。

一旦实验”看起来有价值”，立即升级到完整 DoD。

5.6.2 阻碍 2：”满足 DoD 太花时间了”

回应：一次性投入 30 分钟，换来的是：

- 审稿时不用临时补救（省几天）；
- 后续改进有明确基线（省重复劳动）；
- 论文投稿不用担心”复现不出来”（省心理负担）。

实践建议：

- 把 DoD 检查脚本化，减少手工操作；
- 在 Git pre-commit hook 里加入检查（见第 7 章）；
- 熟练后，DoD 会融入日常流程，不再是额外开销。

5.6.3 阻碍 3：”我们已经有实验管理工具了（如 MLflow、W&B）”

回应：工具很好，但 DoD 是标准，不是工具。

工具可以帮你：

- 自动记录 run 信息（省时间）；

- 可视化实验结果（易于对比）；
- 存储模型和产物（方便管理）。

但工具不能替代：

- 你对” 什么叫完成” 的定义；
- 你对” 评估是否公平” 的检查；
- 你对” 代码是否可复现” 的验证。

建议：把 DoD 清单和工具结合，比如在 MLflow 的 run 里记录” DoD 满足情况” 字段。

5.7 10 分钟动作：为当前最好的结果做 DoD 检查

如果你现在只做一件事：为你目前” 最有希望” 的实验结果做一次完整的 DoD 检查。

1. 找到这次实验的 **run_id**（如果没有，现在就创建一个）
2. 检查 **run** 记录

```
# 查看是否有 run.json
ls outputs/<run_id>/run.json

# 如果没有，立即补救：
# 1. 记录 git commit: git log -1 --format="%H"
# 2. 记录 config 文件路径
# 3. 记录 seed（如果记得）
```

- # 4. 记录数据版本（查看数据目录或日志）
- # 5. 记录环境：`pip freeze > requirements_<run_id>.txt`

3. 尝试复现

- # 切换到记录的 commit
`git checkout <commit_hash>`
- # 用记录的 config 重跑
`python train.py --config <config_path> --seed <seed>`
- # 检查结果是否在合理范围内

4. 记录检查结果

在 `outputs/<run_id>/dod_check.md` 里记录：

- ☐ 记录齐全？
- ☐ 可复现？（结果差异：_____）
- ☐ 评估公平？
- ☐ 有测试？
- ☐ 图表可生成？

5. 如果发现问题，立即修复

- 记录不全：补充 `run.json`
- 无法复现：排查差异，重新跑
- 评估不公平：统一评估脚本，重新跑所有对比实验

完成这个检查后，你会对这个结果的可信度有清晰的认识。如果通过了 DoD，你可以放心地写进论文；如果没有通过，趁现在修复还来得及。

记住：早发现问题，比论文提交前发现要好；论文提交前发现，比审稿时发现要好；审稿时发现，比论文发表后被质疑要好。

6 实验记录自动化：缺的不是工具，是默认行为

6.1 故事引入：三个月后的”考古工作”

论文被接收了，但审稿人要求提供补充材料，解释表 4 中某个实验的具体设置。你打开代码仓库，开始”考古”：

第一步：找日志

你记得这个实验是三个月前跑的，打开 `outputs/` 目录，看到一堆以日期命名的文件夹。但问题是，你记不清具体日期了。你只能一个一个打开，看里面的结果是不是那个实验。

第二步：找配置

终于找到了结果文件，但没有记录配置。你翻遍代码历史，试图找到当时用的超参数。你在某个 `commit` 里找到了可能的配置，但不确定是不是最终版本——你记得当时临时改过学习率，但不确定改成了多少。

第三步：找数据

代码里写的数据路径是 `data/v2/`，但你现在的数据目录是 `data/v3/`。你不记得当时是不是换过数据版本。你在聊天记录里搜索”数据”，试图找到线索。

第四步：放弃

折腾了一个下午，你决定重新跑一次这个实验。但由于参数不确定，重跑的结果和论文里报告的不一致。你只能在补充材料里写：“由于时间久远，部分实验细节可能有偏差。”

审稿人的回复：“We cannot accept a paper where the authors cannot reproduce their own results.”

这个悲剧本可以避免。如果你在实验结束时花 2 分钟记录关键信息，就不会有三个月后的噩梦。

问题不是缺少工具（MLflow、W&B、TensorBoard 都很好），而是缺少记录的默认行为：很多人觉得“这次只是试试，不用记录”，结果试着试着就忘了记录，最后有价值的实验也没留下痕迹。

6.2 两层日志策略：机器精确 + 人类简洁

实验记录的核心挑战是平衡两个需求：

- 机器需要完整精确的信息（用于复现和自动化分析）；
- 人类需要简洁可读的摘要（用于快速回顾和决策）。

如果只有机器日志（如 JSON），人类很难快速了解“这次实验到底想验证什么”；如果只有人类日志（如笔记），机器无法自动复现和对比。

解决方案：两层日志，各司其职。

6.2.1 第一层：机器日志（run.json）

目的：为复现和自动化提供完整、结构化的信息。

原则：

- 自动生成：不依赖人工输入，通过脚本自动收集；
- 结构化：JSON 格式，便于程序解析和查询；
- 完整性：包含复现所需的所有关键信息。

最小字段集（可直接照抄）：

```
{
  "run_id": "2026-02-01_1630_ablation_lr",
  "timestamp": {
    "start": "2026-02-01T16:30:45",
    "end": "2026-02-01T18:45:12"
  },
  "git": {
    "commit": "a1b2c3d4e5f6",
    "dirty": false,
    "branch": "exp/ablation-lr",
    "remote": "git@github.com:user/project.git"
  },
  "config": {
    "path": "configs/ablation_lr.yaml",
    "hash": "sha256:abcd1234...",
    "resolved": {
      "model": "transformer",
      "learning_rate": 3e-4,
      "batch_size": 32,
      ...
    }
  },
}
```



```
"data": {
  "name": "dataset_v3",
  "path": "/data/project/v3",
  "hash": "sha256:ef567890...",
  "split": {
    "train": 8000,
    "val": 1000,
    "test": 1000
  }
},
"environment": {
  "python": "3.11.7",
  "cuda": "12.1",
  "platform": "Linux-5.15.0-x86_64",
  "gpu": "NVIDIA A100-SXM4-40GB",
  "pip_freeze_hash": "sha256:12345678..."
},
"random": {
  "seed": 42,
  "torch_seed": 42,
  "numpy_seed": 42,
  "python_seed": 42
},
"metrics": {
  "val_loss": 0.123,
  "val_acc": 0.943,
  "test_loss": 0.145,
  "test_acc": 0.931,
  "training_time_hours": 2.25
},
"artifacts": {
```

```
"model": "outputs/2026-02-01_1630_ablation_lr/model.pt",
"logs": "outputs/2026-02-01_1630_ablation_lr/train.log",
"predictions": "outputs/2026-02-01_1630_ablation_lr/
predictions.json",
"plots": "outputs/2026-02-01_1630_ablation_lr/plots/"
}
}
```

关键字段解释：

- **run_id**: 唯一标识，建议用时间戳 + 简短描述（见第 2 章）。
- **git.commit**: 代码版本，用 `git rev-parse HEAD` 获取。
- **git.dirty**: 是否有未提交的修改，用 `git diff-index --quiet HEAD --` 检查。如果为 `true`，建议保存 diff: `git diff > changes.patch`。
- **config.resolved**: 展开后的最终配置，包含所有默认值。这很重要，因为默认值可能随代码更新而变化。
- **data.hash**: 数据版本的哈希值，确保数据完全一致。可以用 `sha256sum` 计算整个数据目录的哈希，或使用 DVC 等工具。
- **environment.pip_freeze_hash**: 依赖版本的哈希，用 `pip freeze | sha256sum` 计算。避免完整保存 `pip freeze` 输出（太长），只保存哈希和原始文件路径。
- **random.seed**: 所有随机数种子。确保设置了 PyTorch、NumPy、Python 内置 `random` 的种子。

6.2.2 第二层：人类日志 (run.md)

目的：为人类（包括未来的你）提供快速理解实验的摘要。

原则：

- 简洁：不超过 10 行，核心信息一目了然；
- 结构化：用固定的五要素组织；
- 手写：允许主观判断和洞察。

五要素模板（5 行就够）：

```
# Run: 2026-02-01_1630_ablation_lr
```

```
## 假设 (Hypothesis)
```

测试学习率对收敛速度和最终性能的影响。预期：较小的学习率 ($1e-4$)

```
## 变化 (Change)
```

相比 baseline ($lr=3e-4$)，降低学习率至 $1e-4$ ，其他超参数不变。

```
## 结论 (Result)
```

- 收敛更慢（从 50 epoch 到 80 epoch）
- 最终性能略有提升 (val_acc: 0.943 vs 0.938)
- 训练更稳定，loss 曲线无明显震荡

```
## 下一步 (Next)
```

尝试中间值 $2e-4$ ，可能平衡速度和性能。

考虑 learning rate warmup 策略。

```
## 风险/异常 (Risk)
```

无明显异常。数据增强可能需要配合调整（当前是固定的）。

为什么只要 5 行？

- 降低记录门槛：如果要写一篇长文档，你会拖延；5 行的话，2 分钟就能完成。
- 强制提炼核心：逼迫你思考“这次实验到底验证了什么”，而不是流水账。
- 快速回顾：几个月后再看，5 行摘要比完整日志更有用。

6.3 自动化工具：让记录成为零成本行为

核心理念：记录不应该依赖“记得去做”，而应该自动发生。

6.3.1 在训练脚本中自动生成 run.json

实现示例 (Python)：

```
import json
import subprocess
import hashlib
from pathlib import Path
from datetime import datetime

def log_run(run_id, config, metrics, output_dir):
    """
    自动记录实验信息到 run.json

    Args:
        run_id: 实验唯一标识
        config: 配置字典（已展开）
```

```
metrics: 最终指标字典
output_dir: 输出目录路径

"""
run_info = {
    "run_id": run_id,
    "timestamp": {
        "start": datetime.now().isoformat(),
    },
    "git": get_git_info(),
    "config": {
        "resolved": config,
        "hash": hash_dict(config),
    },
    "data": get_data_info(config.get("data_path")),
    "environment": get_env_info(),
    "random": get_random_seeds(config),
    "metrics": metrics,
    "artifacts": {
        "model": str(output_dir / "model.pt"),
        "logs": str(output_dir / "train.log"),
    }
}

# 保存到文件
run_file = output_dir / "run.json"
with open(run_file, "w") as f:
    json.dump(run_info, f, indent=2)

print(f"Run info logged to {run_file}")

def get_git_info():
```

```
"""获取 git 信息"""
try:
    commit = subprocess.check_output(
        ["git", "rev-parse", "HEAD"]
    ).decode().strip()

    # 检查是否有未提交的修改
    subprocess.check_call(
        ["git", "diff-index", "--quiet", "HEAD", "--"]
    )
    dirty = False
except subprocess.CalledProcessError:
    dirty = True

branch = subprocess.check_output(
    ["git", "rev-parse", "--abbrev-ref", "HEAD"]
).decode().strip()

remote = subprocess.check_output(
    ["git", "config", "--get", "remote.origin.url"]
).decode().strip()

return {
    "commit": commit,
    "dirty": dirty,
    "branch": branch,
    "remote": remote
}

def get_data_info(data_path):
    """获取数据信息"""
```

```
data_path = Path(data_path)

# 计算数据目录哈希（简化版，实际可用 DVC）
# 这里只是示例，真实场景建议用专门的数据版本管理工具

return {
    "name": data_path.name,
    "path": str(data_path.absolute()),
    # "hash": compute_dir_hash(data_path), # 可选
}

def get_env_info():
    """获取环境信息"""
    import sys
    import platform

    env = {
        "python": sys.version.split()[0],
        "platform": platform.platform(),
    }

    # 获取 CUDA 版本（如果可用）
    try:
        import torch
        env["cuda"] = torch.version.cuda
        env["pytorch"] = torch.__version__
        if torch.cuda.is_available():
            env["gpu"] = torch.cuda.get_device_name(0)
    except ImportError:
        pass
```

```
# 保存 pip freeze 到单独文件
pip_freeze = subprocess.check_output(
    ["pip", "freeze"]
).decode()
pip_file = Path("requirements_freeze.txt")
pip_file.write_text(pip_freeze)

env["pip_freeze_hash"] = hashlib.sha256(
    pip_freeze.encode()
).hexdigest()[:16]

return env

def get_random_seeds(config):
    """提取随机种子"""
    return {
        "seed": config.get("seed", None),
        "torch_seed": config.get("torch_seed", None),
        "numpy_seed": config.get("numpy_seed", None),
    }

def hash_dict(d):
    """计算字典的哈希"""
    import json
    return hashlib.sha256(
        json.dumps(d, sort_keys=True).encode()
    ).hexdigest()[:16]
```

在训练脚本中使用：

```
# train.py
```



```
import argparse
from pathlib import Path
from run_logger import log_run # 上面的工具

def main():
    args = parse_args()

    # 创建 run_id 和输出目录
    run_id = f"{datetime.now().strftime('%Y-%m-%d_%H%M')}"
    _{args.exp_name}"
    output_dir = Path("outputs") / run_id
    output_dir.mkdir(parents=True, exist_ok=True)

    # 加载和展开配置
    config = load_config(args.config)
    config = resolve_config(config, args) # 展开所有默认值

    # 设置随机种子
    set_random_seeds(config["seed"])

    # 训练模型
    model, metrics = train_model(config, output_dir)

    # 自动记录实验信息
    log_run(
        run_id=run_id,
        config=config,
        metrics=metrics,
        output_dir=output_dir
    )
```

```
# 提示写 run.md
print(f"\n{' '*60}")
print(f"[OK] Experiment completed: {run_id}")
print(f"[NOTE] Please write a brief summary in:")
print(f"      {output_dir / 'run.md'}")
print(f"\n{' '*60}\n")

if __name__ == "__main__":
    main()
```

6.3.2 用模板简化 run.md 编写

在 output 目录自动生成 run.md 模板：

```
def create_run_md_template(output_dir, run_id):
    """创建 run.md 模板"""
    template = f"""# Run: {run_id}
```

```
## 假设 (Hypothesis)
[这次实验想验证什么？ 预期结果是什么？ ]
```

```
## 变化 (Change)
[相比之前的实验，改了什么？ ]
```

```
## 结论 (Result)
[实验结果如何？ 有什么意外发现？ ]
```

```
## 下一步 (Next)
[基于这次结果，下一步打算做什么？ ]
```

```
## 风险/异常 (Risk)
```

```
[有没有什么值得注意的异常或风险? ]
```

```
"""
```

```
md_file = output_dir / "run.md"
if not md_file.exists():
    md_file.write_text(template)
    print(f"[NOTE] run.md template created at {md_file}")
```

这样，每次实验结束后，你只需要填空，而不是从零开始写。

6.4 与现有工具集成

6.4.1 与 MLflow 集成

如果你已经在使用 MLflow，可以把 run.json 信息同步到 MLflow：

```
import mlflow
```

```
def log_to_mlflow(run_info):
```

```
    """将 run.json 信息记录到 MLflow"""
```

```
    with mlflow.start_run(run_name=run_info["run_id"]):
```

```
        # 记录参数
```

```
        mlflow.log_params(run_info["config"]["resolved"])
```

```
        # 记录指标
```

```
        mlflow.log_metrics(run_info["metrics"])
```

```
        # 记录环境信息
```

```
        mlflow.log_dict(run_info["environment"], "environment.j
```

```
# 记录 git 信息
mlflow.set_tag("git.commit", run_info["git"]["commit"])
mlflow.set_tag("git.branch", run_info["git"]["branch"])
mlflow.set_tag("git.dirty", run_info["git"]["dirty"])

# 记录产物
mlflow.log_artifact(run_info["artifacts"]["model"])
```

6.4.2 与 Weights & Biases 集成

```
import wandb

def log_to_wandb(run_info):
    """将 run.json 信息记录到 W&B"""
    wandb.init(
        project="my-research",
        name=run_info["run_id"],
        config=run_info["config"]["resolved"],
        tags=[run_info["git"]["branch"]]
    )

    # 记录指标
    wandb.log(run_info["metrics"])

    # 记录环境和 git 信息
    wandb.config.update({
        "git_commit": run_info["git"]["commit"],
        "git_dirty": run_info["git"]["dirty"],
        "python_version": run_info["environment"]["python"],
```

```
}))
```

```
# 保存模型
```

```
wandb.save(run_info["artifacts"]["model"])
```

重点：工具是辅助，核心是记录的规范。即使没有 MLflow/W&B，有 run.json 和 run.md 也足够。

6.5 日志查询与分析

有了结构化的 run.json，你可以快速查询和对比实验：

6.5.1 查找最好的实验

```
# find_best_run.py
```

```
import json
```

```
from pathlib import Path
```

```
def find_best_runs(metric="test_acc", top_k=5):
```

```
    """查找指标最好的实验"""
```

```
    runs = []
```

```
    for run_dir in Path("outputs").iterdir():
```

```
        if not run_dir.is_dir():
```

```
            continue
```

```
        run_json = run_dir / "run.json"
```

```
        if not run_json.exists():
```

```
            continue
```

```
with open(run_json) as f:
    run_info = json.load(f)

if metric in run_info.get("metrics", {}):
    runs.append({
        "run_id": run_info["run_id"],
        metric: run_info["metrics"][metric],
        "config": run_info["config"]["resolved"]
    })

# 排序
runs.sort(key=lambda x: x[metric], reverse=True)

print(f"Top {top_k} runs by {metric}:")
for i, run in enumerate(runs[:top_k], 1):
    print(f"{i}. {run['run_id']}: {run[metric]:.4f}")
    print(f"    Config: lr={run['config'].get('learning_rate')}
          f"bs={run['config'].get('batch_size')}")

return runs[:top_k]

if __name__ == "__main__":
    find_best_runs()
```

6.5.2 对比两个实验的配置差异

```
# compare_runs.py
```

```
import json
```

```
from pathlib import Path

def compare_runs(run_id1, run_id2):
    """对比两个实验的配置和结果"""
    run1 = load_run(run_id1)
    run2 = load_run(run_id2)

    print(f"Comparing {run_id1} vs {run_id2}\n")

    # 对比配置
    config1 = run1["config"]["resolved"]
    config2 = run2["config"]["resolved"]

    print("Configuration differences:")
    for key in set(config1.keys()) | set(config2.keys()):
        val1 = config1.get(key, "N/A")
        val2 = config2.get(key, "N/A")
        if val1 != val2:
            print(f"  {key}: {val1} -> {val2}")

    # 对比指标
    print("\nMetrics:")
    metrics1 = run1.get("metrics", {})
    metrics2 = run2.get("metrics", {})
    for key in set(metrics1.keys()) & set(metrics2.keys()):
        val1 = metrics1[key]
        val2 = metrics2[key]
        diff = val2 - val1
        print(f"  {key}: {val1:.4f} -> {val2:.4f} "
              f"({diff:+.4f}, {diff/val1*100:+.2f}%)")
```

```
def load_run(run_id):
    """加载实验信息"""
    run_json = Path("outputs") / run_id / "run.json"
    with open(run_json) as f:
        return json.load(f)

if __name__ == "__main__":
    import sys
    if len(sys.argv) != 3:
        print("Usage: python compare_runs.py <run_id1> <run_id2>")
        sys.exit(1)

    compare_runs(sys.argv[1], sys.argv[2])
```

6.6 常见问题与解决方案

6.6.1 Q1: run.json 太大了怎么办?

问题：如果保存完整的 config（包含模型定义、数据预处理细节等），run.json 可能很大。

解决方案：

- 只保存”关键超参数”在 run.json 里（如学习率、batch size）；
- 完整 config 保存到单独文件：config_resolved.yaml；
- run.json 里只记录 config 文件路径和哈希。

6.6.2 Q2: 忘了记录 run.md 怎么办?

解决方案：

- 在 Makefile 或脚本里加入检查：

```
if [ ! -f outputs/$RUN_ID/run.md ]; then
    echo "Warning: run.md not found for $RUN_ID"
    echo "Please write a summary before continuing."
fi
```

- 定期（如每周五）检查哪些实验缺少 run.md，集中补充。
- 实在想不起来就写”遗忘”，总比没有记录强。

6.6.3 Q3：数据版本太大，无法计算哈希怎么办？

解决方案：

- 使用数据版本管理工具（如 DVC、Git LFS）；
- 或者只记录数据的”清单文件”（manifest）：

```
# 生成清单
find data/ -type f | xargs sha256sum > data_manifest.txt

# 计算清单哈希
sha256sum data_manifest.txt
```

- 在 run.json 里记录清单文件路径和哈希。

6.7 10 分钟动作：为下一个实验建立自动记录

如果你现在只做一件事：建立自动记录的最小系统。

1. 复制 **run_logger.py**

把前面的 `log_run` 函数复制到你的项目里。

2. 在训练脚本末尾调用

```
# 训练结束后
log_run(run_id, config, metrics, output_dir)
```

3. 创建 **run.md** 模板

```
create_run_md_template(output_dir, run_id)
```

4. 跑一次实验测试

跑一次实验，确认：

- `outputs/<run_id>/run.json` 自动生成了
- `outputs/<run_id>/run.md` 模板生成了
- `run.json` 里的信息完整（`git`、`config`、`env`、`metrics`）

5. 填写 **run.md**

花 2 分钟填写五要素模板。

从下一个实验开始，记录就是自动的、零成本的。你唯一要做的就是花 2 分钟写 5 行摘要——这个投入会在三个月后得到百倍回报。

7 AI 编码助手加入后， workflow 必须升级

7.1 故事引入：当 AI 从加速器变成地雷制造机

你用了一个月时间，在 Copilot 的帮助下快速搭建了一个复杂的多模态学习框架。代码生成速度惊人——以前一天能写 200 行，现在半天就能写 500 行。你觉得自己的生产力翻了几倍。

但在准备论文的前一周，问题开始集中爆发：

问题 1：隐藏的 Bug 你在测试一个边界情况时，发现模型输出了 NaN。追溯代码，发现是一个数据预处理函数有问题——它在某些罕见情况下会除以零。这个函数是 Copilot 生成的，你当时只检查了“能跑”，没有仔细审查边界条件。

问题 2：不一致的实现 你发现数据增强在训练和评估时的实现不一致：训练用的是 Copilot 生成的版本 A，评估用的是你后来手写的版本 B，两者在归一化方式上有微妙差异。这导致训练和测试的分布不匹配，性能受到影响。

问题 3：过度复杂的架构 为了“优雅”，你让 Copilot 帮你设计了一个高度抽象的架构，有很多层抽象类和工厂模式。现在你需要快速修改一个功能，却发现要改动 5 个文件，因为逻辑分散在各

个抽象层里。

问题 4：缺失的验证 你意识到很多 Copilot 生成的代码都没有测试覆盖。你一直依赖”跑通就行”，从没系统验证过边界情况、错误处理、数据一致性。现在想补测试，却发现代码耦合度太高，难以单独测试。

悔悟时刻 你不得不停下所有新功能开发，花了一周时间：

- 审查所有 AI 生成的代码，找出隐藏问题；
- 统一不一致的实现；
- 简化过度设计的架构；
- 补充缺失的测试和验证。

你意识到：**AI 编码助手不是问题，问题是你的 workflow 没有跟上。**

在 AI 加入之前，你的代码量少，错误少，可以靠记忆和经验把控质量。但现在代码量暴增，而你的验证机制还停留在”手工审查”阶段，这个缺口会越来越大。

7.2 AI 编码的三大陷阱

7.2.1 陷阱 1：生成速度超过验证能力

症状：AI 一分钟生成 100 行代码，你觉得”看起来没问题”就接受了。但你其实没有仔细检查边界条件、错误处理、性能影响。

后果：

- 埋下大量隐藏 bug，在关键时刻暴露；
- 代码库充斥着”能跑但不健壮”的实现；
- 重构成本远超一开始就做对的成本。

本质问题：验证机制没有升级，还停留在”手工审查”阶段。

7.2.2 陷阱 2：重复生成导致不一致

症状：你需要类似功能时，让 AI 重新生成，而不是复用现有代码。结果项目里有多”类似但不完全相同”的实现。

后果：

- 修改一个逻辑需要改多处，容易遗漏；
- 不同实现可能有微妙差异，导致结果不一致；
- 代码库膨胀，维护成本暴增。

本质问题：没有区分”核心库代码”和”一次性胶水代码”（回顾第 3 章）。

7.2.3 陷阱 3：过度依赖”顺手改一下”

症状：在让 AI 实现一个功能时，你顺便让它”把这个也改了”、”把那个也优化了”。一个 commit 包含了十几个文件的改动。

后果：

- 无法定位哪个改动引起了问题；
- 想要回滚时发现”牵一发而动全身”；

- 代码历史混乱，无法追溯逻辑演进。

本质问题：改动不再是”可验证的最小单元”（回顾第 1 章）。

7.3 升级后的 workflow：把 AI 当成”快速试错的初级程序员”

核心理念：AI 是助手，不是替代品。它的优势是快速生成初稿，你的责任是验证、整合、把关。

把 AI 编码看作两阶段过程：

1. 生成阶段：AI 快速产出初稿（快速、粗糙、允许有问题）；
2. 验证阶段：人类审查、测试、重构（慢速、严格、确保质量）。

关键原则：生成可以快，但上线必须慢。

7.3.1 原则 1：每次只做一个可验证的改动

操作建议：

- 每次让 AI 生成的代码不超过 200 行 diff；
- 每次改动只涉及一个逻辑功能（如”添加数据增强”、”修复 NaN bug”）；
- 改动必须能独立验证（有对应的测试或实验）。

实践案例：

反例：

一次改动包含：

- 修改数据加载逻辑
- 添加新的模型层
- 调整训练超参数
- 重构评估代码
- 更新配置文件

结果：如果实验结果变差，你无法确定是哪个改动导致的。

正例：

改动 1（独立 commit）：

- 修改数据加载逻辑
- 测试：数据加载测试通过，输出 shape 正确

改动 2（独立 commit）：

- 添加新的模型层
- 测试：模型前向传播测试通过

改动 3（独立 commit）：

- 运行实验，对比新模型和 baseline
- 记录：run_id, metrics, 结论

7.3.2 原则 2：每次改动附带“如何验证”

操作建议：

在让 AI 生成代码时，同时要求它生成验证代码。

示例提示词：

请实现数据增强功能，包括：

1. 实现代码
2. 单元测试（至少覆盖正常情况和边界情况）
3. 使用示例
4. 可能的风险点

验证清单（每次改动后检查）：

- ☐ 代码能运行（不崩溃）
- ☐ 输出符合预期（shape、数值范围、数据类型）
- ☐ 边界情况处理正确（空输入、极端值、错误输入）
- ☐ 不影响现有功能（回归测试通过）
- ☐ 有测试覆盖（至少 smoke test）

7.3.3 原则 3：禁止”顺手重构一大片”

症状识别：

危险信号：

- AI 提议：“我顺便帮你重构一下这部分代码，让它更优雅”
- 你想：“反正都要改，不如一起做了”
- 结果：一个 PR 包含功能添加 + 重构 + bug 修复

正确做法：

1. 先实现功能，后重构：

Step 1: 实现功能（允许粗糙）

```
git commit -m "feat: add data augmentation (rough version)"
```


Step 2: 验证功能正确

```
pytest tests/test_data_aug.py
```

Step 3: 独立重构（不改功能）

```
git commit -m "refactor: clean up data augmentation code"
```

Step 4: 验证重构没有破坏功能

```
pytest tests/test_data_aug.py
```

2. 重构与功能分离：

- 功能改动：commit message 用”feat.” 或”fix.”
- 重构改动：commit message 用”refactor.”
- 绝不混在一起

7.3.4 原则 4：核心逻辑必须人工审查

定义”核心逻辑”：

以下代码不能盲目接受 AI 生成的版本，必须人工仔细审查：

- 数据处理：数据加载、预处理、划分、增强
- 模型核心：损失函数、关键模块（注意力、归一化等）
- 评估逻辑：指标计算、后处理、阈值选择
- 随机性控制：随机种子设置、随机采样逻辑
- 超参数：学习率调度、权重衰减、dropout 率

审查清单：

- ☐ 逻辑正确性（算法实现对吗？）
- ☐ 边界条件（空数据、极端值会怎样？）
- ☐ 数据一致性（训练和测试逻辑一致吗？）
- ☐ 随机性（种子设置正确吗？可复现吗？）
- ☐ 性能影响（会不会特别慢？内存会爆吗？）

7.3.5 原则 5：先补最小测试，再改核心逻辑

反面模式：

你：Copilot，帮我重构这个数据加载函数

Copilot：[生成 150 行代码]

你：看起来不错，接受！

[一周后发现数据加载有问题，但已经运行了很多实验]

正确流程：

Step 1: 先为现有代码写测试

```
def test_data_loader_current():
```

```
    """测试当前数据加载行为"""
```

```
    loader = DataLoader(...)
```

```
    batch = next(iter(loader))
```

```
    assert batch['image'].shape == (32, 3, 224, 224)
```

```
    assert batch['label'].shape == (32,)
```

```
    assert batch['label'].min() >= 0
```

```
    assert batch['label'].max() < num_classes
```

Step 2: 运行测试，确保通过

```
pytest tests/test_data_loader.py
```

```
# Step 3: 进行重构
```

```
[Copilot 生成新代码]
```

```
# Step 4: 运行测试，确保行为不变
```

```
pytest tests/test_data_loader.py
```

```
# Step 5: 如果测试失败，修复或回滚
```

测试先行的好处：

- 明确当前行为（作为回归测试基准）
- 快速验证重构（1 分钟而不是 1 小时实验）
- 避免破坏现有功能
- 强制思考”正确的行为是什么”

7.4 AI 规则页 (CLAUDE.md): 团队协作的共识

为了在团队中统一 AI 使用规范，建议在项目根目录创建 CLAUDE.md（或 AI_RULES.md），明确 AI 编码的边界和流程。

7.4.1 CLAUDE.md 模板

```
# AI 编码助手使用规范
```

本项目使用 AI 编码助手（如 GitHub Copilot、Claude、GPT）辅助开发。为确保代码质量和可维护性，所有 AI 生成的代码必须遵循以下规范。

核心原则

1. ****AI 生成，人类负责****：AI 可以快速产出初稿，但人类必须审查、测试、把关。
2. ****小步快跑****：每次改动不超过 200 行 diff，必须可独立验证。
3. ****测试先行****：核心逻辑改动前，先补充测试；改动后，测试必须通过。
4. ****禁止顺手重构****：功能改动与重构必须分开 commit。

操作规范

1. 每次改动必须附带验证方式

在 commit message 或 PR 描述里明确：

...

如何验证此改动：

- [] 运行测试： `pytest tests/test_xxx.py`
- [] 运行实验： `python train.py --config xxx`
- [] 检查输出：输出 shape 应为 [B, C, H, W]

...

2. 每次改动必须记录风险点

...

潜在风险：

- 修改了数据预处理，可能影响数据分布
- 新增了随机操作，需要检查种子设置
- 改动了评估逻辑，需要重新跑 baseline 确认公平性

...

3. 每次改动必须有回滚策略

...

回滚策略：

- 如果测试失败： `git checkout -- <文件>`
- 如果实验结果变差：回退到 `commit <hash>`
- 如果影响其他功能：回滚并新建 `branch` 单独调试

...

禁止的行为

[NO] **禁止 1：盲目接受大段生成代码**

- 任何超过 50 行的 AI 生成代码都必须人工逐行审查
- 核心逻辑（数据、模型、评估）必须特别严格审查

[NO] **禁止 2：改动太多文件**

- 单次 `commit` 不应超过 5 个文件（特殊情况除外）
- 如果需要改很多文件，拆分成多个 `commit`

[NO] **禁止 3：跳过测试直接合并**

- 任何代码合并到 `main` 之前，必须通过所有测试
- 如果没有相关测试，必须先补充测试

[NO] **禁止 4：在 `main` 分支直接改动**

- 所有 AI 生成的代码必须在实验分支上先验证
- 验证通过后再合并到 `main`

必须的行为

[YES] **必须 1：核心逻辑有测试覆盖**

- 数据加载、模型核心、评估逻辑必须有单元测试
- 每次修改核心逻辑后，测试必须通过

[YES] **必须 2：实验结果可对比**

- 任何影响结果的改动，必须运行对比实验
- 记录改动前后的 `run_id` 和指标对比

[YES] **必须 3：配置版本同步**

- 任何影响结果的改动，必须更新对应的 `config`
- `Config` 文件必须 `commit` 并打 `tag`

[YES] **必须 4：文档同步更新**

- API 改动必须更新文档
- 新功能必须更新 `README` 的使用说明

验收标准（合并到 `main` 前）

- [] 改动不超过 200 行（或经过充分讨论）
- [] 所有测试通过
- [] 核心逻辑经过人工审查
- [] 有清晰的 `commit message`
- [] 附带验证方式、风险点、回滚策略
- [] 如影响实验结果，附带对比实验记录

示例：好的 AI 辅助流程

```
```bash
```

```
1. 创建实验分支
```

```
git checkout -b exp/add-mixup
```

```
2. 让 AI 生成初稿
```

```
[Copilot 生成 mixup 数据增强代码]
```

```
3. 人工审查和修改
```

```
- 检查边界条件
```

```
- 添加类型注解
```

```
- 确认随机种子处理正确
```

```
4. 补充测试
```

```
pytest tests/test_mixup.py
```

```
5. 运行对比实验
```

```
make train CONFIG=configs/baseline.yaml # baseline
```

```
make train CONFIG=configs/mixup.yaml # with mixup
```

```
6. 记录结果
```

```
outputs/2026-02-01_1030_baseline/run.json
```

```
outputs/2026-02-01_1100_mixup/run.json
```

```
7. 如果结果改进，合并到 main
```

```
git checkout main
```

```
git merge exp/add-mixup
```

```
8. 清理实验分支
```

```
git branch -d exp/add-mixup
```

```
...
```

```
参考资源
```

- 仓库结构规范：见 README.md "项目结构" 章节
- DoD 清单：见 DoD.md
- 实验记录规范：见 LOGGING.md

---

**\*\*原则总结：AI 让你更快，但不能让你更马虎。\*\***

## 7.5 实战案例：AI 辅助的正确打开方式

### 7.5.1 案例 1：添加新模型组件

错误方式：

你：Copilot，帮我实现一个多头注意力模块

Copilot：[生成 200 行代码]

你：看起来不错，接受！直接用在训练里

[训练崩溃，发现注意力计算有 bug]

正确方式：

# Step 1: 生成初稿

你：Copilot，帮我实现一个多头注意力模块，包括：

- 实现代码
- 单元测试
- 使用示例

# Step 2: 人工审查

- 检查 softmax 前是否有除以  $\sqrt{d_k}$
- 检查 mask 的实现是否正确
- 检查输出 shape 是否符合预期

# Step 3: 写测试



```
def test_multi_head_attention():
 attn = MultiHeadAttention(d_model=512, n_heads=8)
 x = torch.randn(32, 100, 512) # (batch, seq, dim)
 out, weights = attn(x, x, x)

 assert out.shape == (32, 100, 512)
 assert weights.shape == (32, 8, 100, 100)
 assert torch.allclose(weights.sum(dim=-1),
 torch.ones_like(weights.sum(dim=-1)))

Step 4: 独立测试模块
pytest tests/test_attention.py -v

Step 5: 在小数据上测试集成
python train.py --config configs/test_attention.yaml \
 --data_subset 100 --epochs 2

Step 6: 确认没问题后，正式训练
python train.py --config configs/attention.yaml
```

### 7.5.2 案例 2: 重构数据加载

错误方式:

你: 这个数据加载代码太乱了, Copilot 帮我重构

Copilot: [重写了整个 DataLoader 类]

你: 好, 用新版本!

[重新跑实验, 结果和之前不一致, 不知道哪里出错了]

正确方式:

# Step 1: 为旧版本写行为测试

```
def test_old_data_loader_behavior():
 """记录旧版本的行为作为基准"""
 loader = OldDataLoader(...)
 batch = next(iter(loader))

 # 记录关键行为
 assert batch['image'].shape == (32, 3, 224, 224)
 assert batch['image'].dtype == torch.float32
 assert batch['image'].min() >= -1.0
 assert batch['image'].max() <= 1.0
 # ... 更多断言
```

# Step 2: 运行测试，确保通过

```
pytest tests/test_data_loader.py::test_old_behavior -v
```

# Step 3: 让 AI 重构

Copilot: [生成新的 DataLoader]

# Step 4: 为新版本写相同的测试

```
def test_new_data_loader_behavior():
 """确保新版本行为一致"""
 loader = NewDataLoader(...)
 batch = next(iter(loader))

 # 相同的断言
 assert batch['image'].shape == (32, 3, 224, 224)
 # ...
```

# Step 5: 对比两个版本的输出

```
def test_output_consistency():
```

```
"""直接对比新旧版本输出"""
old_loader = OldDataLoader(seed=42)
new_loader = NewDataLoader(seed=42)

old_batch = next(iter(old_loader))
new_batch = next(iter(new_loader))

torch.testing.assert_close(old_batch['image'],
 new_batch['image'])

Step 6: 如果一致，再运行完整实验验证
make reproduce RUN=baseline # 用旧版本
make train CONFIG=configs/baseline_new_loader.yaml # 用新版本
对比结果
```

## 7.6 常见问题与解决方案

### 7.6.1 Q1: AI 生成的代码太复杂，看不懂怎么办？

解决方案：

- 拒绝接受！让 AI 重新生成更简单的版本。
- 提示词：

请实现 xxx 功能，要求：

- 代码简单直接，避免过度抽象
- 尽量使用标准库和常见模式
- 添加详细注释解释每个步骤

- 记住：你看不懂的代码，未来的你也看不懂，别人更看不懂。

## 7.6.2 Q2: AI 生成的代码有 bug，但改了又出现新 bug

解决方案：

- 不要反复让 AI 修 bug，陷入无限循环。
- 正确做法：
  1. 先写测试，复现 bug
  2. 人工修复 bug（小范围改动）
  3. 测试通过后，再考虑是否需要重构
- AI 可以提示可能的问题，但修复应该由人来做。

## 7.6.3 Q3: 团队成员不遵守 AI 使用规范怎么办?

解决方案：

- 在 Git pre-commit hook 里加入自动检查：

```
.git/hooks/pre-commit
#!/bin/bash

检查 commit diff 行数
DIFF_LINES=$(git diff --cached | wc -l)
if [$DIFF_LINES -gt 400]; then
 echo "Error: commit diff too large ($DIFF_LINES lines)"
 echo "Please split into smaller commits"
 exit 1
```

```
fi

检查是否有测试
if git diff --cached --name-only | grep -q "src/"; then
 if ! git diff --cached --name-only | grep -q "tests/"; then
 echo "Warning: you modified src/ but no test changes"
 echo "Please ensure tests are updated"
 fi
fi

运行测试
pytest tests/ -q || {
 echo "Error: tests failed"
 exit 1
}
```

- 在 CI 中强制执行（见下节）
- Code review 时严格检查

## 7.7 10 分钟动作：为下一次 AI 编码建立检查点

如果你现在只做一件事：为下一次 AI 辅助编码建立最小验证流程。

### 1. 创建 **CLAUDE.md**

复制本章的模板，放到项目根目录。

### 2. 写一个 **smoke test**

```
tests/test_smoke.py

def test_basic_training_loop():
 """基本训练流程能跑通"""
 config = load_config("configs/test.yaml")
 config["epochs"] = 2
 config["data_subset"] = 100

 model, metrics = train_model(config)

 assert metrics["train_loss"] > 0
 assert not math.isnan(metrics["val_loss"])
```

### 3. 设置 pre-commit hook

```
简单版本：只检查测试
echo '#!/bin/bash\npytest tests/ -q' > .git/hooks/
pre-commit
chmod +x .git/hooks/pre-commit
```

### 4. 试一次完整流程

```
1. 创建实验分支
git checkout -b exp/test-ai-workflow

2. 让 AI 做一个小改动（如添加一个工具函数）
3. 写对应测试
4. 运行测试
pytest tests/test_xxx.py
```

```
5. Commit (会自动运行 hook)
git add .
git commit -m "feat: add utility function X"
```

```
6. 如果 hook 通过，合并到 main
git checkout main
git merge exp/test-ai-workflow
```

从下一次使用 AI 编码开始，默认行为就是：生成 → 审查 → 测试 → 验证 → 合并。这个流程会成为肌肉记忆，确保 AI 始终是你的提速器，而不是地雷制造机。

## 8 多路径探索如何不变垃圾堆

---

### 8.1 故事引入：从”灵活探索”到”不敢动任何东西”

你的研究项目进行到第三个月。你是一个勤奋的研究者，尝试了很多不同的方向：

- 路径 A：改进模型架构（5 种不同的注意力机制）
- 路径 B：优化训练策略（3 种不同的学习率调度）
- 路径 C：增强数据质量（4 种预处理方式）
- 路径 D：调整损失函数（6 种不同的损失组合）

你很兴奋——这么多探索！肯定能找到有效的组合！

但当你打开项目目录，看到的是这样的景象：

```
experiments/
 train_v1.py
 train_v2.py
 train_v2_fixed.py
 train_v3_final.py
```



```
train_v3_really_final.py
train_attention_test.py
train_loss_ablation.py
... (20+ 个文件)
```

```
outputs/
 run_0523/
 exp_new/
 test_attention/
 final_results/
 final_results_v2/
 backup_0601/
 temp/
... (50+ 个目录)
```

```
configs/
 config.yaml
 config_old.yaml
 config_backup.yaml
 config_test.yaml
... (15+ 个文件)
```

问题开始暴露：

**问题 1：找不到最好的结果** 你记得某次实验效果很好，但你不记得是哪个配置、哪个输出目录。你开始一个一个打开目录，查看日志，试图找到那个结果。花了两个小时，还是不确定找对了没有。

**问题 2：不敢删除任何东西** outputs/ 已经占了 50GB 空间，但你不敢删除任何目录——万一删掉的正好是论文需要的那个实验呢？你决定”先留着，反正硬盘够大”。

**问题 3：无法对比不同路径** 你想对比”路径 A（注意力改进）”和”路径 B（学习率优化）”的效果，但你发现：

- 它们用的 baseline 不一样（一个是三个月前的，一个是最近的）
- 评估脚本不一样（一个算的是 top-1，一个是 top-5）
- 数据划分可能也不一样（记不清了）

**问题 4：无法合并有效改进** 你在路径 A 里发现了一个有效的改进，想把它移植到路径 B 里，但发现：

- 路径 A 和路径 B 的代码已经分化了
- 数据加载逻辑不兼容
- 合并需要大量人工工作

你意识到：灵活探索变成了无序混乱，多路径并行变成了垃圾堆。

## 8.2 为什么多路径探索容易失控

研究的本质就是不确定性：你不知道哪条路会成功，所以需要同时探索多个方向。但如果没有管理机制，探索的越多，混乱度就越高。

### 8.2.1 失控的三个阶段

**阶段 1：快速探索（第 1-4 周） 行为：**

- 想到什么就试什么，不拘泥于规范
- 代码复制粘贴，改个名字就用
- 输出随便找个地方放，”先跑起来再说”

**感觉：**充满活力，进展快速。

**阶段 2：路径分化（第 5-8 周） 行为：**

- 不同路径的代码开始分化，共同部分减少
- 每条路径有自己的数据处理、训练脚本、评估方式
- 新想法基于某个旧路径，而不是主线

**感觉：**有点乱，但还能记住大致情况。

**阶段 3：失控混乱（第 9 周 +） 行为：**

- 完全记不住哪个实验是哪个路径的
- 不敢删除任何东西，空间占用暴增
- 想要合并改进时发现路径已经完全不兼容
- 准备论文时重新跑实验，结果对不上记忆

**感觉：**焦虑、无力、想推倒重来。

## 8.2.2 根本原因：缺少”可丢弃”和”可合并”机制

多路径探索的核心挑战是：

- 不知道哪条路会成功，所以需要并行探索多条路径
- 不能全部保留，否则会淹没在信息海洋里
- 成功的路径需要合并回主线，否则无法形成完整方案

如果缺少管理机制：

- 路径无法安全丢弃（怕删错）
- 路径无法轻松合并（代码分化）
- 路径无法清晰对比（条件不一致）

## 8.3 核心机制：隔离 + 可丢弃 + 可对比

### 8.3.1 机制 1：每条路径必须隔离

隔离的三要素：

#### 1. 独立的 Git 分支

<code>exp/path-A-attention</code>	# 路径 A：注意力改进
<code>exp/path-B-lr-schedule</code>	# 路径 B：学习率优化
<code>exp/path-C-data-aug</code>	# 路径 C：数据增强
<code>exp/path-D-loss-combo</code>	# 路径 D：损失组合

好处：

- 代码改动相互独立，不会冲突
- 可以随时切换、对比、合并
- Git 历史清晰记录每条路径的演进

## 2. 独立的配置文件

```
configs/
 baseline.yaml # 公共基准
 path_A_attention.yaml # 路径 A 的配置
 path_B_lr_schedule.yaml # 路径 B 的配置
 path_C_data_aug.yaml # 路径 C 的配置
 path_D_loss_combo.yaml # 路径 D 的配置
```

配置里明确继承关系：

```
path_A_attention.yaml
base: baseline.yaml # 继承基准配置

只列出差异
model:
 attention_type: "multi_head" # 改动点
 num_heads: 8

experiment:
 name: "path_A_attention"
 hypothesis: "多头注意力比单头更有效"
```

## 3. 独立的输出目录

```
outputs/
```

```
path_A/
 2026-02-01_1030_baseline/
 2026-02-01_1500_multi_head_attn/
 2026-02-02_0900_improved_attn/
path_B/
 2026-02-01_1100_baseline/
 2026-02-01_1600_cosine_schedule/
 2026-02-02_1000_warmup_schedule/
path_C/
 ...
```

好处:

- 每条路径的实验结果清晰分组
- 删除整个路径时，只需删除对应目录
- 归档时可以按路径打包

### 8.3.2 机制 2: 明确的生命周期管理

每条探索路径都应该有明确的生命周期:

创建 → 探索 → 评估 → 决策 (保留/归档/删除)

#### 创建阶段

##### # 1. 创建分支

```
git checkout main
```

```
git checkout -b exp/path-E-new-idea
```

##### # 2. 创建配置

```
cp configs/baseline.yaml configs/path_E_new_idea.yaml
编辑配置，记录假设
```

```
3. 创建输出目录
```

```
mkdir -p outputs/path_E/
```

```
4. 记录路径信息
```

```
cat > outputs/path_E/README.md <<EOF
```

```
路径 E：新想法探索
```

```
假设
```

```
[这条路径要验证什么假设？]
```

```
基准对比
```

```
对比基准：outputs/baseline/2026-02-01_1030_baseline
```

```
预期改进：[预期能提升多少？]
```

```
关键改动
```

```
- [改动 1]
```

```
- [改动 2]
```

```
开始时间
```

```
2026-02-05
```

```
状态
```

```
探索中
```

```
EOF
```

探索阶段 在分支上自由迭代，记录每次实验：

```
运行实验
```

```
python train.py --config configs/path_E_new_idea.yaml \
 --output outputs/path_E/2026-02-05_1030_try1/
```

# 记录结果 (run.json 自动生成, run.md 手写)

# 见第6章

# 继续迭代

# 每次实验用新的 run\_id, 不覆盖之前的

评估阶段 定期 (如每周) 评估路径价值:

# 评估清单

## 效果评估

- 最好结果: [指标]
- 对比基准: [改进幅度]
- 稳定性: [多次运行的方差]

## 成本评估

- 时间成本: [训练时间增加了多少?]
- 计算成本: [需要更多资源吗?]
- 复杂度成本: [代码复杂度增加了多少?]

## 洞察收获

- 发现了什么? [即使没成功, 有什么学到的?]
- 失败原因: [为什么没达到预期?]
- 副产品: [有没有意外收获?]

## 决策

- [ ] 继续探索 (值得深入)
- [ ] 合并到主线 (成功)



[ ] 归档（有价值但不是当前重点）

[ ] 删除（无价值）

决策阶段 根据评估结果，做出清晰决策：

决策 1：合并到主线（路径成功）

# 1. 整理代码

# 确保改动最小、干净、可测试

# 2. 运行完整验证

```
make test
```

```
make reproduce RUN=path_E/best_result
```

# 3. 合并

```
git checkout main
```

```
git merge exp/path-E-new-idea
```

# 4. 打 tag

```
git tag -a milestone-E-success -m \
```

```
"路径 E 成功：新想法将 baseline 性能从 x 提升到 y"
```

# 5. 更新 baseline

```
cp outputs/path_E/best_result outputs/baseline/
```

# 6. 删除实验分支

```
git branch -d exp/path-E-new-idea
```

# 7. 更新路径状态

```
echo "Status: Merged to main (2026-02-12)" >> outputs/
path_E/README.md
```

**决策 2：归档（有价值但非当前重点）**

# 1. 打 tag 保存分支状态

```
git tag -a archive/path-E-v1 -m \
"路径 E 归档：初步有效但需要更多时间验证"
```

# 2. 整理产物

```
mkdir -p archives/path_E/
cp -r outputs/path_E/ archives/path_E/
cp configs/path_E/*.yaml archives/path_E/
```

# 3. 写总结

```
cat > archives/path_E/SUMMARY.md <<EOF
路径 E 归档总结
```

## 主要发现

[总结关键发现]

## 为什么归档

[解释为什么现在不继续，但值得保留]

## 未来重启条件

[什么情况下值得重新探索？]

## 参考资料

- 代码版本: `git tag archive/path-E-v1`
- 最好结果: `outputs/path_E/2026-02-10_1500_best/`
- 相关论文: [外部参考]

EOF

# 4. 删除实验分支（保留 tag）

```
git branch -d exp/path-E-new-idea
```

```
5. 删除 outputs (已归档)
```

```
rm -rf outputs/path_E/
```

**决策 3: 删除 (无价值)**

```
1. 最后确认
```

```
检查是否有任何有价值的发现或代码
```

```
2. 删除输出
```

```
rm -rf outputs/path_E/
```

```
3. 删除配置
```

```
rm configs/path_E_*.yaml
```

```
4. 删除分支
```

```
git branch -D exp/path-E-new-idea # -D 强制删除
```

```
5. 记录删除原因 (可选但推荐)
```

```
cat >> docs/EXPLORATION_LOG.md <<EOF
```

```
路径 E (已删除, 2026-02-12)
```

```
- 假设: [原始假设]
```

```
- 结果: [为什么失败]
```

```
- 教训: [学到了什么]
```

```
EOF
```

### 8.3.3 机制 3: 公平对比的基准线

所有路径对比时, 必须使用同一基准:

## 建立基准实验

# 1. 在 main 分支跑基准实验

```
git checkout main
```

```
python train.py --config configs/baseline.yaml \
 --output outputs/baseline/2026-02-01_1030_base
```

# 2. 验证基准可复现

```
make reproduce RUN=baseline/2026-02-01_1030_baseline
```

# 3. 打 tag

```
git tag -a baseline-v1 -m "Common baseline for all paths"
```

# 4. 记录基准信息

```
cat > outputs/baseline/INFO.md <<EOF
```

# 基准实验信息

## 配置

- Config: configs/baseline.yaml
- Commit: \$(git rev-parse HEAD)
- Tag: baseline-v1

## 结果

- Val accuracy: 0.920
- Test accuracy: 0.915
- Training time: 2.5 hours

## 用途

所有路径 (A-Z) 的对比基准。

任何路径的改进都应该相对这个基准来报告。

## 复现

```
make reproduce RUN=baseline/2026-02-01_1030_baseline
EOF
```

## 路径对比标准化

# 对比脚本示例

# compare\_paths.py

```
import json
```

```
from pathlib import Path
```

```
def compare_to_baseline(path_name):
```

```
 """对比某条路径和基准的结果"""
```

```
 baseline = load_best_run("outputs/baseline")
```

```
 path = load_best_run(f"outputs/{path_name}")
```

```
 print(f"\n{'='*60}")
```

```
 print(f"路径对比: {path_name} vs Baseline")
```

```
 print(f"{'='*60}\n")
```

# 对比配置差异

```
print("配置差异: ")
```

```
diff_configs(baseline["config"], path["config"])
```

# 对比指标

```
print("\n指标对比: ")
```

```
compare_metrics(baseline["metrics"], path["metrics"])
```

# 对比成本

```
print("\n成本对比: ")
```

```
compare_cost(baseline, path)

结论
print("\n结论: ")
if is_improvement(path["metrics"], baseline["metrics"]):
 print(f"[OK] 路径 {path_name} 成功改进 baseline")
 print(f" 建议: 合并到主线")
else:
 print(f"[NO] 路径 {path_name} 未能改进 baseline")
 print(f" 建议: 归档或删除")

if __name__ == "__main__":
 import sys
 compare_to_baseline(sys.argv[1])
```

## 8.4 每周清理仪式：实验墓地整理

核心理念：定期清理是避免垃圾堆的唯一方法。

### 8.4.1 每周五下午的清理流程（30 分钟）

**Step 1:** 列出所有活跃路径（5 分钟）

```
list_active_paths.sh

echo "活跃的探索路径: "
git branch | grep "exp/" | while read branch; do
 echo " - $branch"
done
```

```
echo "\n输出目录大小: "
du -sh outputs/*/ | sort -rh
```

**Step 2: 逐个评估路径 (15 分钟)** 对每条路径问三个问题:

1. 这周有新进展吗?
  - 有: 继续保留
  - 没有: 是暂停还是放弃?
2. 相比 **baseline** 有改进吗?
  - 有: 达到合并标准了吗?
  - 没有: 还有继续价值吗?
3. 占用多少资源?
  - 输出目录大小
  - 代码复杂度
  - 维护成本

**Step 3: 执行清理动作 (10 分钟)**

```
清理脚本示例
weekly_cleanup.sh

#!/bin/bash

echo "开始每周清理..."

1. 归档两周前的路径 (如果有 tag)
git tag -l "archive/*" | while read tag; do
```

```

 tag_date=$(git log -1 --format=%ai $tag | cut -d' ' -f1)
 # [归档逻辑]
done

2. 删除标记为 "to_delete" 的输出
find outputs/ -name ".to_delete" -type f | while read marker; do
 dir=$(dirname $marker)
 echo "删除: $dir"
 rm -rf $dir
done

3. 压缩一个月前的输出（如果还有价值）
find outputs/ -type d -mtime +30 | while read dir; do
 if [-f "$dir/run.json"]; then
 echo "压缩: $dir"
 tar -czf "${dir}.tar.gz" $dir
 rm -rf $dir
 fi
done

4. 报告空间释放
echo "\n清理完成！"
du -sh outputs/

```

### 8.4.2 清理决策树

对于每条路径，判断：

+-+ 过去两周有活动？

|



```

+-- 是 -> 相比 baseline 有改进?
|
|
| +-- 是(>5%) -> [合并到主线]
| +-- 是(3-5%) -> [继续观察]
| +-- 否(<3%) -> [考虑放弃]
|
+-- 否 -> 是否有归档价值?
|
| +-- 是 (有独特洞察) -> [归档]
| +-- 否 -> [删除]

```

特殊情况：

- 占用空间 >10GB -> 优先处理（压缩或删除）
- 有外部引用（如论文草稿）-> 暂不删除，打标记
- 代码复杂度高 -> 如无明显价值，倾向删除

## 8.5 路径合并策略：从探索到稳定主线

### 8.5.1 合并前检查清单

在合并路径到 main 之前，确保满足：

- [ ] 相比 baseline 有稳定改进（多次运行均有效）
- [ ] 改动最小化（只保留必要的修改）
- [ ] 代码干净可维护（通过 lint 和 review）
- [ ] 有测试覆盖（至少 smoke test）
- [ ] 配置清晰记录（可复现）
- [ ] 不破坏现有功能（回归测试通过）
- [ ] 文档已更新（README、API docs）

## 8.5.2 渐进式合并策略

对于复杂路径，不要一次性全部合并。推荐分步骤：

示例：合并”路径 A：注意力改进”

# 路径 A 包含三个改动：

# 1. 新的注意力机制

# 2. 改进的位置编码

# 3. 调整的学习率

# 不要一次性合并所有改动！

# Step 1: 先合并最核心的改进（注意力）

```
git checkout main
```

```
git checkout exp/path-A-attention -- src/models/attention.py
```

```
git commit -m "feat: add improved attention mechanism from path A"
```

# 验证

```
make test
```

```
make train CONFIG=configs/main_with_new_attention.yaml
```

# Step 2: 如果 Step 1 成功，再合并位置编码

```
git checkout exp/path-A-attention -- src/models/position_encoding.py
```

```
git commit -m "feat: add improved position encoding from path A"
```

# 验证

```
make test
```

```
make train CONFIG=configs/main_with_attention_and_pos.yaml
```

# Step 3: 最后合并超参数调整

# [如果前两步都成功的话]

好处:

- 每一步都可独立验证
- 如果某步失败，不影响其他改进
- Git 历史清晰记录每个改进
- 便于定位问题

## 8.6 常见问题与解决方案

### 8.6.1 Q1: 路径太多了，记不住怎么办?

解决方案：维护一个路径追踪表。

# docs/EXPLORATION\_TRACKER.md

# 探索路径追踪

路径	状态	假设	最好结果	决策	更新时间
A-attention	进行中	多头注意力更有效	0.925 (+0.5%)	继续	
B-lr-schedule	归档	Cosine 调度更好	0.922 (+0.2%)	效果	
C-data-aug	进行中	MixUp 能提升泛化	0.930 (+1.0%)	**考	
D-loss-combo	删除	多任务损失有帮助	0.918 (-0.2%)	反向	
E-new-idea	刚启动	[待验证]	-	探索	2026-02-12

## 基准

Baseline: 0.920 (outputs/baseline/2026-02-01\_1030\_baseline)

## ## 下周计划

- 路径 A: 完成消融实验, 确认每个组件贡献
- 路径 C: 多跑几个 `seed` 验证稳定性
- 路径 E: 初步实现和验证

每周更新这个表格 (5 分钟), 就能清晰掌控所有路径状态。

## 8.6.2 Q2: 不同路径的代码冲突怎么办?

预防胜于治疗:

- 路径尽量改不同的模块 (如一个改数据、一个改模型)
- 共享的核心代码放在 `src/`, 不轻易修改
- 路径特有的改动放在 `experiments/` 里

冲突发生时:

- 不要强行合并多个路径
- 先合并一个, 验证成功后, 再基于新 `main` 重新创建其他路径
- 或者: 重新评估是否真的需要合并多个路径

## 8.6.3 Q3: 删除路径后后悔了怎么办?

预防措施:

- 删除前打 `tag`:

```
git tag -a deleted/path-X -m "Path X before deletion"
```

- 删除前写简短总结（见上文“删除决策”）
- 重要的数据先归档到便宜的存储（如云端）

恢复方法：

```
如果有 tag，可以恢复代码
```

```
git checkout deleted/path-X
```

```
基于此重新创建分支
```

```
git checkout -b exp/path-X-restored
```

```
如果输出被删除了，检查归档或备份
```

```
ls archives/path_X/
```

## 8.7 10 分钟动作：整理当前的探索路径

如果你现在只做一件事：清点并分类当前所有探索路径。

### 1. 列出所有分支和输出

```
git branch | grep "exp/"
```

```
ls outputs/
```

### 2. 为每条路径快速分类

在笔记里写下：

路径 A (exp/xxx): [进行中 | 归档 | 删除]

- 假设:

- 状态:

- 决策:

路径 B (exp/yyy): [进行中 | 归档 | 删除]

- ...

### 3. 执行一次清理

# 删除明确无价值的路径

```
git branch -D exp/failed-path-X
```

```
rm -rf outputs/path_X/
```

# 归档有价值但不活跃的路径

```
git tag -a archive/path-Y
```

```
mkdir -p archives/path_Y/
```

```
mv outputs/path_Y/ archives/path_Y/
```

# 为活跃路径更新状态记录

### 4. 建立追踪表

创建 docs/EXPLORATION\_TRACKER.md, 记录所有活跃路径。

### 5. 设定下周清理时间

在日历里加入: 每周五 17:00 - 探索路径清理 (30 分钟)

完成这个 10 分钟动作后, 你会立刻感觉到:

- 对项目状态的掌控力增强

- 知道哪些路径值得继续、哪些该放弃
- 不再担心”垃圾堆”会失控

记住：多路径探索是研究的必要特征，但不管理的多路径探索会变成灾难。定期清理不是负担，而是保持清醒的必要仪式。

## 9 让最后阶段不爆炸的 3 个提前动作

---

### 9.1 故事引入：离 **deadline** 只剩一周的噩梦

周一早上，你查看日历，心里一沉——论文截稿倒计时：7 天。

你原本计划这周只是”最后润色”：把实验结果整理成图表，写好相关工作，检查一遍格式。应该很轻松，对吧？

但当你开始准备论文时，问题像雪崩一样涌来：

周一：发现主实验无法复现 你想重新跑一次主实验，确认数字没记错。但运行脚本后，结果和三周前不一样——准确率从 94.3% 降到了 92.1%。

你慌了。开始排查：

- 是代码变了吗？Git 历史一片混乱，你不确定当时用的哪个版本。
- 是数据变了吗？数据目录里有 v1、v2、v3，记不清了。
- 是环境变了吗？某个依赖库自动升级了？

你花了一整天也没找到原因。



周二：发现 **baseline** 不公平 审稿人肯定会关注你和 baseline 的对比。你仔细检查，发现一个致命问题：你的方法用的是最新的数据预处理，但 baseline 用的是旧版本。评估口径根本不一致。

你需要重新跑 baseline——但这需要 6 小时训练时间。

周三：发现缺少关键的消融实验 导师看了你的初稿，指出：“你的方法包含三个改进（A、B、C），但你没有说明每个改进贡献了多少。审稿人肯定会问。”

你意识到缺少消融实验（ablation study）。你需要跑：

- baseline
- baseline + A
- baseline + B
- baseline + C
- baseline + A + B
- baseline + A + C
- baseline + B + C
- baseline + A + B + C（完整方法）

每个实验 2 小时，8 个实验 = 16 小时。但你只有 4 天了。

周四：发现图表数据找不到 你想生成论文图表，但发现关键实验的输出文件找不到了——可能被你不小心删了，或者在某次清理时弄丢了。你只记得“结果很好”，但原始数据没了。

你不得不重新跑那些实验。

周五：开始怀疑人生 你已经三天没睡好觉了。实验还在跑，论文还没开始写，图表还没做完。你开始怀疑：“为什么我总是在最后阶段爆炸？”

答案很简单：因为你没有提前做三件事。

## 9.2 为什么“最后阶段爆炸”几乎是必然

回顾本书第一章，我们说过研究中有三种债务：

- 探索债：代码混乱、输出散乱、路径不清
- 验证债：baseline 不强、消融缺失、对比不公平
- 复现债：环境不固定、配置不全、版本不明

如果这些债务在日常积累，最后阶段就是集中偿还期。而 deadline 的压力会放大一切问题：

- 平时可以慢慢调试的问题，现在必须立刻解决
- 平时可以重跑的实验，现在没时间了
- 平时可以请教的问题，现在别人也忙

最残酷的真相：如果你在最后一周才发现问题，大概率已经来不及修复了。

那怎么办？答案是：提前暴露问题，提前解决，或者至少提前知道有问题。

## 9.3 提前动作 1: 每周一次”可复现性自查”(15 分钟)

### 9.3.1 为什么重要

核心理念：你不能等到论文截稿前才发现结果无法复现。必须在日常就持续验证可复现性。

如果每周都做一次自查，问题会在出现的当周被发现，而不是累积到最后。

### 9.3.2 自查清单 (15 分钟完成)

**第 1 项：检查本周最重要的实验能否复现 (5 分钟)**

# 找到本周最好/最重要的实验

`RUN_ID="本周最好的 run_id"`

# 检查记录完整性

[ ] `outputs/$RUN_ID/run.json` 存在

[ ] `run.json` 里有 `git commit`

[ ] `run.json` 里有 `config` 路径

[ ] `run.json` 里有 `seed`

[ ] `run.json` 里有数据版本

[ ] `run.json` 里有环境信息

# 如果任何一项缺失，立刻补救

**第 2 项：尝试快速复现 (5 分钟)**    不需要完整重跑 (太慢)，但要验证流程能跑通：

# 用小数据集快速测试

```
python train.py \
 --config outputs/$RUN_ID/config.yaml \
 --data_subset 100 \
 --epochs 2 \
 --seed 42
```

# 检查:

- [ ] 能正常启动
- [ ] 数据加载正确
- [ ] 模型前向传播正确
- [ ] 损失计算正常
- [ ] 评估流程正确

如果这 2 分钟的测试都跑不通, 说明完整复现肯定有问题。现在修复还来得及。

### 第 3 项: 检查依赖是否漂移 (3 分钟)

# 保存当前依赖

```
pip freeze > requirements_$(date +%Y%m%d).txt
```

# 对比上周的依赖

```
diff requirements_上周日期.txt requirements_$(date +%Y%m%d).txt
```

# 如果有变化, 记录到 CHANGELOG.md

依赖变化是复现问题的常见原因。每周记录可以在问题出现时快速定位。

### 第 4 项: 检查输出是否有标记 (2 分钟)

```
检查本周输出是否都有 run_id
ls outputs/
```

```
检查是否有"unnamed"、"temp"、"test"这类临时目录
如果有，要么删除，要么给它们起正式名字
```

无名输出是”未来的坑”——你现在知道它是什么，但一个月后就忘了。

### 9.3.3 自查频率与时机

推荐时间：每周五下午最后 15 分钟

为什么是周五？

- 一周工作结束，可以全面回顾
- 周末不工作的话，可以安心休息（知道项目状态可控）
- 如果发现问题，下周一可以立刻处理

特殊情况：

- 出现”看起来不错”的结果时：立刻做自查，不要拖到周五
- 改了核心代码后：当天就做自查
- 切换数据版本后：立刻做自查

### 9.3.4 常见陷阱

陷阱 1：”反正都记得，不用检查” 现实：两周后你就会忘记细节。记忆不可靠，记录才可靠。

**陷阱 2：**”这次只是测试，不用记录” 现实：很多”只是测试”的实验后来成了论文主结果，但当时没记录，最后追悔莫及。

**陷阱 3：**”反正能跑，应该能复现” 现实：”能跑”和”能在另一台机器/另一个环境/两个月后复现”是完全不同的事。

## 9.4 提前动作 2：每月一次”债务盘点” (30 分钟)

### 9.4.1 为什么重要

周自查解决的是”最近的实验能否复现”，但还有更深层的问题：

- 整个项目有多少探索债？
- 有多少验证债？
- 有多少复现债？

每月盘点就是强制你抬头看全局，而不是一直埋头做实验。

### 9.4.2 盘点清单 (30 分钟完成)

#### 探索债盘点 (10 分钟)

##### # 1. 统计代码混乱度

```
git ls-files | wc -l # 总文件数
git ls-files | grep "test\|tmp" | wc -l # 临时文件数
git log --oneline | head -20 # 最近 20 次 commit
```

##### # 2. 统计输出混乱度

```
du -sh outputs/ # 总大小
```

```
ls outputs/ | wc -l # 目录数
find outputs/ -name "run.json" | wc -l # 有记录的实验数
```

### # 3. 计算探索债指标

有记录实验 / 总目录数 = 记录覆盖率

健康标准:

- 记录覆盖率 >80%: 良好
- 记录覆盖率 60-80%: 警戒
- 记录覆盖率 <60%: 危险 (需要立刻清理)

### 验证债盘点 (10 分钟)

# 检查验证完整性

论文候选结果清单:

```
[] 主实验 (Table 2) → run_id: _____
[] Baseline 对比 (Table 3) → run_id: _____
[] 消融实验 (Table 4) → run_id: _____
[] 失败案例分析 (Figure 5) → run_id: _____
```

对于每个结果:

- [ ] 有完整 run.json
- [ ] 有 baseline 对比 (公平评估)
- [ ] 有多次运行 (不是偶然)
- [ ] 有测试覆盖 (smoke test 通过)

健康标准:

- 所有论文候选结果都有 run\_id: 良好
- 缺少 1-2 个: 警戒 (下月补齐)
- 缺少 3 个以上: 危险 (论文写不了)

### 复现债盘点 (10 分钟)

# 找到最重要的 3 个实验

```
TOP_3_RUNS="..."
```

# 对每个实验做复现测试

```
for run_id in $TOP_3_RUNS; do
 echo "Testing $run_id..."
```

# 检查记录

```
[-f outputs/$run_id/run.json] || echo " 缺少 run.json"
```

# 快速复现测试 (小数据)

```
python train.py \
 --config outputs/$run_id/config.yaml \
 --data_subset 100 --epochs 2 \
 || echo " 快速复现失败"
```

# 依赖检查

```
pip install -r outputs/$run_id/requirements.txt \
 || echo " 依赖可能有变化"
```

```
done
```

健康标准:

- 3 个都能快速复现: 良好



- 2 个能复现：警戒
- 1 个或 0 个能复现：危险（需要紧急修复）

### 9.4.3 债务可视化

建议维护一个” 债务趋势图”：

```
debt_tracking.csv
```

月份,探索债(记录覆盖率),验证债(候选结果完整性),复现债(可复现比例)

2026-01,50%,60%,33%

2026-02,70%,80%,67%

2026-03,85%,100%,100%

如果债务在累积（数字下降），说明你在”透支未来”。如果债务在减少（数字上升），说明你在”偿还债务”。

目标：论文截稿前三个月，所有债务指标应该 >90%。

## 9.5 提前动作 3：论文前三个月建立”复现基线” (1 小时)

### 9.5.1 为什么重要

最大的误区：认为”论文写作阶段”才需要考虑复现性。

现实：如果你等到写论文时才开始准备复现材料，会发现：

- 很多实验细节已经忘了
- 代码版本对不上

- 数据找不到了
- 环境变了

正确做法：在”实验阶段”就建立复现基线，论文阶段只需要验证和补充。

## 9.5.2 复现基线的内容

最小复现包（1 小时建立）

```
reproduce/
 README.md # 复现指南
 environment.yaml # 环境定义
 data_manifest.txt # 数据清单
 baseline_runs.txt # 关键实验列表
 reproduce.sh # 一键复现脚本
 verify.py # 验证脚本
```

### README.md 模板

```
复现指南

环境准备（10 分钟）

```bash
# 创建环境
conda env create -f environment.yaml
conda activate research-env

# 验证安装
```

```
python verify.py --check-env
...

## 数据准备 (30 分钟)

```bash
下载数据 (需要约 5GB 空间)
bash scripts/download_data.sh

验证数据
python verify.py --check-data
...

复现关键实验 (6 小时)

```bash
# 复现主实验 (Table 2, 约 2 小时)
make reproduce RUN=main_experiment
# 预期结果: accuracy 94.3%  $\pm$  0.5%

# 复现 baseline (Table 3, 约 2 小时)
make reproduce RUN=baseline
# 预期结果: accuracy 92.0%  $\pm$  0.3%

# 复现消融实验 (Table 4, 约 2 小时)
bash scripts/reproduce_ablation.sh
...

## 验证结果

```bash
```

# 自动验证所有结果

```
python verify.py --check-results
```

# 输出应该显示:

```
Main experiment: within expected range
```

```
Baseline: within expected range
```

```
Ablation: all components verified
```

```
...
```

## 故障排除

见 docs/TROUBLESHOOTING.md

### **verify.py 示例**

```
import json
```

```
from pathlib import Path
```

```
def verify_environment():
```

```
 """验证环境是否正确"""
```

```
 import torch
```

```
 print(f" PyTorch version: {torch.__version__}")
```

```
 print(f" CUDA available: {torch.cuda.is_available()}")
```

```
 # 更多检查...
```

```
def verify_data():
```

```
 """验证数据是否完整"""
```

```
 data_manifest = Path("data_manifest.txt").read_text()
```

```
 # 检查文件是否存在、哈希是否一致...
```

```
 print(" Data verification passed")
```

```
def verify_results(run_id, expected_metric, tolerance=0.01):
 """验证结果是否在预期范围内"""
 run_json = Path(f"outputs/{run_id}/run.json")
 with open(run_json) as f:
 run_info = json.load(f)

 actual = run_info["metrics"]["test_acc"]
 diff = abs(actual - expected_metric)

 if diff <= tolerance:
 print(f" {run_id}: {actual:.3f} "
 f"(expected {expected_metric:.3f} ± {tolerance:.3f})")
 return True
 else:
 print(f" {run_id}: {actual:.3f} "
 f"(expected {expected_metric:.3f}, diff {diff:.3f})")
 return False

if __name__ == "__main__":
 import argparse
 parser = argparse.ArgumentParser()
 parser.add_argument("--check-env", action="store_true")
 parser.add_argument("--check-data", action="store_true")
 parser.add_argument("--check-results", action="store_true")
 args = parser.parse_args()

 if args.check_env:
 verify_environment()
 if args.check_data:
 verify_data()
 if args.check_results:
```

```
验证所有关键实验
verify_results("main_experiment", expected_metric=0.943)
verify_results("baseline", expected_metric=0.920)
...
```

### 9.5.3 建立时机

最佳时机：当你有第一个“看起来能写进论文”的结果时，立刻建立复现基线。

不要等到：

- ☐ 实验都做完了
- ☐ 开始写论文了
- ☐ 准备投稿了

应该是：

- ☐ 有第一个有希望的结果（即使还不完美）
- ☐ 确定大致的技术路线
- ☐ 能回答“这个项目最终要证明什么”

经验法则：论文截稿前 3 个月建立复现基线。如果是会议论文（6 个月项目），则在第 3 个月建立。

## 9.6 紧急补救方案：如果已经到了最后阶段

### 9.6.1 如果距离 **deadline** 只剩 2 周

接受现实：你没时间“做对一切”了。必须聚焦最重要的事。

### 优先级 1：确保主结果可复现（3 天）

```
Day 1: 找到主实验的代码版本
- 通过 Git 历史、聊天记录、笔记回忆
- 找到最接近的 commit
- 补充 run.json（尽可能回忆参数）

Day 2: 在干净环境重跑
- 新建虚拟环境
- 记录所有依赖
- 重新运行，记录结果

Day 3: 如果无法精确复现
- 如果差异在 1-2% 内：可以接受，注明误差
- 如果差异更大：在论文里诚实说明原因
- 最坏情况：改用可复现的次优结果
```

### 优先级 2：补最关键的验证（2 天） 只补审稿人肯定会问的验证：

- 如果只能选一个：补 baseline 公平对比
- 如果能选两个：再补主要的消融实验
- 其他的：可以说“due to time constraints, left for future work”

### 优先级 3：写最小复现文档（1 天）

```
最小复现文档包含：
1. 环境说明（Python 版本、关键库版本）
2. 数据获取方式（链接或联系方式）
3. 运行命令（哪怕只有一个）
```

4. 预期结果（数值范围）
5. 已知问题（诚实说明复现困难）

## 9.6.2 如果距离 **deadline** 只剩 1 周

残酷的真相：你已经没时间重跑实验了。只能尽力补救记录。

# 能做的（每项 2 小时）：

- [ ] 为所有论文实验补充 `run.json`（尽可能回忆）
- [ ] 把现有代码打上 `Git tag`（保存当前状态）
- [ ] 写最简单的复现说明（README 里一节）
- [ ] 把所有输出文件打包备份（防止丢失）

# 不要做的（没时间了）：

- [ ] 不要试图重跑所有实验
- [ ] 不要试图建立完美的复现环境
- [ ] 不要试图修复所有不一致

心态调整：接受不完美，但确保最低可追溯性。论文能投出去比完美复现更重要。

## 9.6.3 事后补救

如果论文被接收，要求提供代码：

# 你有 2-4 周时间补救

Week 1：回溯和记录

- 找到所有论文相关的代码版本
- 尽力复现关键结果



- 记录所有"不一致"的地方

Week 2-3: 清理和验证

- 清理代码（删除不相关部分）
- 补充文档和注释
- 确保至少 1-2 个结果能复现

Week 4: 打包和发布

- 把代码整理成可发布形式
- 写清晰的 README
- 在论文里诚实说明复现的限制

## 9.7 10 分钟动作：今天就做的自查

如果你现在只做一件事：对当前项目做一次最小自查。

### 1. 找到最重要的实验（1 分钟）

问自己：如果只能保留一个实验结果，是哪个？

写下它的 `run_id`（如果没有，现在就给它创建一个）

### 2. 检查记录完整性（3 分钟）

[ ] 有 `run.json` 吗？

[ ] 知道用的哪个 `Git commit` 吗？

[ ] 知道用的哪个 `config` 吗？

[ ] 知道随机种子吗？

[ ] 知道数据版本吗？

任何一项缺失，立刻补救（写在笔记里也行）

### 3. 快速复现测试（5 分钟）

```
用小数据测试流程能否跑通
python train.py \
 --config <你的config> \
 --data_subset 100 \
 --epochs 2
```

如果报错，记录错误信息，下次工作时优先修复

### 4. 设定下次自查时间（1 分钟）

在日历里加入：

- 每周五 17:00：可复现性自查（15 分钟）
- 每月最后一天：债务盘点（30 分钟）
- [项目开始 + 3个月]：建立复现基线（1 小时）

完成这个 10 分钟自查后，你会得到两个重要成果：

1. 心里有底：你知道项目的核心成果是可追溯的
2. 提前预警：如果发现问题，你还有时间修复

## 9.8 本章总结：预防胜于救火

最后阶段爆炸的根本原因是：把验证推迟到了最后才做。

正确的心态是：

- 不要等”确定能用”才记录——任何”看起来不错”的结果都要立刻记录
- 不要等”写论文”才验证复现性——日常就要持续验证
- 不要等”审稿人要求”才补实验——提前识别验证债，主动补齐

三个提前动作是你的保险：

1. 每周自查：确保最近的工作可追溯（15 分钟）
2. 每月盘点：确保债务不失控（30 分钟）
3. 提前建立复现基线：确保最后阶段不手忙脚乱（1 小时）

总计每月投入时间： $15 \text{ 分钟} \times 4 + 30 \text{ 分钟} + 1 \text{ 小时}$ （首次）= 2.5 小时

这 2.5 小时的投入，可以避免最后阶段的 **3 天到 3 周** 的救火时间。

记住：研究的不确定性不可避免，但最后阶段的爆炸是可以预防的。

## 10 带学生/合作者的版本： 团队最小规范

---

### 10.1 故事引入：从”各自为战”到”互相拖累”

你的研究团队有三个人：你（博士生）、一个硕士生学弟、一个本科生实习生。你们在做同一个项目，应该是合作关系。

但现实是这样的：

周一早上的站会 你：“我周末跑了新模型，效果不错，准确率95%。”

学弟：“太好了！能发我代码吗？我想基于这个继续改。”

你：“呃……代码在我电脑上，有点乱，我整理一下再发你。”  
(实际上你根本不知道从哪里开始整理)

实习生：“我上周做的数据预处理，你们用了吗？”

你和学弟：“呃……用的哪个版本？”

实习生：“就是我发在群里的那个链接……”（三人翻聊天记录，找不到）

导师：“你们三个到底在做什么？为什么每个人说的数字都不一样？”

周三的代码冲突 学弟：“师兄，我把代码 push 了，你 pull 一下。”

你 pull 代码，结果：

```
CONFLICT (content): Merge conflict in src/model.py
CONFLICT (content): Merge conflict in configs/default.yaml
CONFLICT (content): Merge conflict in train.py
```

你打开代码一看，学弟几乎改了所有文件。而且很多改动你看不懂——他加了一堆新参数，但没注释。

你花了两个小时解决冲突，最后发现：你的代码被覆盖了，上周跑出的好结果复现不了了。

周五的数据灾难 实习生：“师兄，我不小心把 data/ 目录删了，你那里有备份吗？”

你：“什么？！那个目录有 20GB，是我花了三天处理的数据！”

实习生：“我以为那是临时文件……Git 没有追踪它……”

你和学弟都没有完整备份。你们不得不重新下载原始数据，重新运行三天的预处理流程。

一周过去了，项目不仅没有进展，反而倒退了。

## 10.2 为什么“各自厉害”不等于“团队高效”

团队协作的反直觉之处在于：单个人的能力很强，但团队产出却很低。

### 10.2.1 三大协作陷阱

**陷阱 1：隐性知识依赖** 每个人脑子里都有很多”只有自己知道”的信息：

- 这个参数为什么设成这个值
- 这段代码为什么这么写
- 这个实验为什么失败了
- 这份数据为什么要这样处理

当需要协作时，这些隐性知识成为瓶颈——别人只能”等你有空解释”，而你被反复打断。

**陷阱 2：重复劳动** 没有明确的分工和接口，导致：

- 两个人写了功能相同但实现不同的代码
- 两个人用不同方式处理同样的数据
- 两个人跑了相同的实验但记录方式不同

表面上是”并行工作”，实际上是”浪费算力和时间”。

**陷阱 3：集成成本爆炸** 每个人在自己的分支上”做得很好”，但合并时发现：

- 接口不兼容
- 依赖版本冲突
- 配置方式不统一

- 评估标准不一致

最后花在”合并和对齐”上的时间，比各自开发的时间还多。

## 10.2.2 根本原因：缺少”团队规范”

个人项目可以”随心所欲”——反正只有你自己懂。

但团队项目需要显式规范：

- 代码怎么写
- 实验怎么记录
- 改动怎么合并
- 问题怎么沟通

没有规范 = 每个人都在用自己的方式 = 无法协作。

## 10.3 最小规范 1：代码规范（从混乱到可读）

### 10.3.1 命名规范：让代码自解释

文件命名

# 糟糕的命名

test.py

new.py

model2.py

train\_final.py

# 清晰的命名

<code>train_baseline.py</code>	# 训练基准模型
<code>train_with_attention.py</code>	# 训练带注意力的模型
<code>evaluate_on_testset.py</code>	# 在测试集上评估
<code>preprocess_raw_data.py</code>	# 预处理原始数据

## 变量和函数命名

# 糟糕的命名

```
def f(x, y):
 z = x + y
 return z
```

# 清晰的命名

```
def compute_weighted_loss(prediction, target, weight):
 """
 计算加权损失

 Args:
 prediction: 模型预测 (batch_size, num_classes)
 target: 真实标签 (batch_size,)
 weight: 类别权重 (num_classes,)

 Returns:
 loss: 加权交叉熵损失
 """
 raw_loss = cross_entropy(prediction, target)
 weighted_loss = raw_loss * weight[target]
 return weighted_loss.mean()
```

命名原则：



- 用完整单词，不要缩写（除非是公认缩写如 num, max, avg）
- 函数名用动词开头（compute, load, save, train, evaluate）
- 变量名用名词（model, dataset, config, metrics）
- 布尔变量用 is/has/should 开头（is\_training, has\_attention, should\_save）

### 10.3.2 注释规范：写给未来的你和队友

#### 必须注释的地方

##### 1. 函数文档字符串（每个函数都要有）

```
def train_one_epoch(model, dataloader, optimizer, device):
 """
 训练一个 epoch

 Args:
 model: PyTorch 模型
 dataloader: 训练数据加载器
 optimizer: 优化器
 device: 设备 ('cuda' 或 'cpu')

 Returns:
 avg_loss: 平均损失
 accuracy: 训练准确率
 """
 ...
```

##### 2. 不明显的逻辑

```
对注意力权重进行温度缩放，防止 softmax 饱和
attention_scores = attention_scores / temperature

使用 gradient clipping 防止梯度爆炸
阈值设为 1.0 是基于初步实验确定的
torch.nn.utils.clip_grad_norm_(model.parameters(), max_n
```

### 3. 已知问题和 TODO

```
TODO: 这里的实现效率不高，需要优化
目前 $O(n^2)$ 复杂度，数据量大时会很慢

FIXME: 当 batch size = 1 时会崩溃
临时解决方案：在外层检查并跳过

NOTE: 这个超参数对结果影响很大
改动前务必先在小数据集上测试
```

### 不要注释的地方

```
不要注释显而易见的事
x = x + 1 # 给 x 加 1

不要用注释"解释烂代码"，应该重构
这个函数很复杂，做了很多事情，慢慢看...
def complicated_function():
 ...

重构成清晰的函数
```

```
def load_data():
 ...

def preprocess_data(data):
 ...

def train_model(data):
 ...
```

### 10.3.3 PR 模板：即使一个人也要用

Pull Request（或 Merge Request）模板强制你在合并前回答关键问题。

#### 创建 PR 模板

```
.github/pull_request_template.md
```

#### ## 改动概述

[用一句话描述这个 PR 做了什么]

#### ## 改动类型

- [ ] 新功能 (feature)
- [ ] Bug 修复 (fix)
- [ ] 重构 (refactor)
- [ ] 文档 (docs)
- [ ] 测试 (test)

#### ## 改动详情

[详细说明改了什么、为什么改]

## 如何验证

```
```bash
```

提供验证步骤

```
make test
```

```
python train.py --config configs/test.yaml
```

```
```
```

## 潜在风险

[这个改动可能带来什么问题? ]

## 相关实验

- Run ID: [如果相关, 提供 run\_id]
- 结果对比: [新/旧实现的指标对比]

## Checklist

- [ ] 代码通过 lint 检查
- [ ] 添加了必要的测试
- [ ] 更新了相关文档
- [ ] 所有测试通过
- [ ] 改动不破坏现有功能 (回归测试)

为什么”一个人也要用”?

- 强制你思考”这个改动到底做了什么”
- 为未来的队友 (或未来的你) 留下清晰记录
- 养成习惯后, 团队协作时自然就规范了

## 10.4 最小规范 2：实验规范（从口头到可查）

### 10.4.1 统一 run\_id 命名

问题：每个人用不同方式命名实验，导致混乱。

解决方案：团队统一 run\_id 格式。

# 团队统一格式

YYYY-MM-DD\_HHMM\_<person>\_<experiment>

例如：

2026-02-15\_1030\_zhangsan\_baseline

2026-02-15\_1400\_lisi\_attention\_ablation

2026-02-16\_0900\_wangwu\_data\_augmentation

好处：

- 按时间排序
- 知道是谁的实验（出问题时知道找谁）
- 知道实验内容

### 10.4.2 统一 config 管理

问题：每个人的 config 格式不同，无法对比。

解决方案：团队共享一个 base config，个人只写差异。

configs/

base.yaml

# 团队基准配置

```
people/
 zhangsan_*.yaml # 张三的个人实验配置
 lisi_*.yaml # 李四的个人实验配置
paper/ # 论文相关的"官方"配置
 baseline.yaml
 main_method.yaml
 ablation_*.yaml
```

### **base.yaml 示例**

```
configs/base.yaml
团队基准配置，不要随意修改
如有必要修改，必须在团队会议讨论

model:
 type: "transformer"
 hidden_dim: 512
 num_layers: 6

data:
 path: "/data/shared/project_data_v3"
 batch_size: 32
 num_workers: 4

training:
 epochs: 100
 learning_rate: 3e-4
 optimizer: "adam"

evaluation:
 metric: "accuracy"
```

```
eval_every: 1000
```

### 个人配置示例

```
configs/people/zhangsan_attention_test.yaml
```

```
继承基准配置
```

```
base: ../base.yaml
```

```
只写差异
```

```
model:
```

```
 attention_type: "multi_head" # 改动点
```

```
 num_heads: 8 # 新增参数
```

```
标记实验信息
```

```
experiment:
```

```
 owner: "张三"
```

```
 hypothesis: "多头注意力能提升 2-3% 准确率"
```

```
 related_runs:
```

```
 - "2026-02-15_1030_zhangsan_baseline"
```

### 10.4.3 统一 logging 格式

问题：每个人记录方式不同，无法汇总对比。

解决方案：使用统一的 run.json 和 run.md 模板（见第 6 章）。

团队约定：

1. 每次实验结束后 10 分钟内必须：

- 生成 run.json（自动）

- 填写 `run.md`（手动 5 行）
2. `run.md` 必须填写的字段：
- 假设：这次实验想验证什么
  - 变化：相比什么改了什么
  - 结论：结果如何（一句话）
  - 下一步：基于结果下一步做什么
  - @ 导师：是否需要导师关注
3. 如果忘记记录：
- 当周组会时会被提醒
  - 连续两次忘记：需要补齐所有缺失记录

## 10.5 最小规范 3：沟通规范（从口头到留痕）

### 10.5.1 周会规范：只看可复现的 `run`

问题：每周组会变成”口头汇报比赛”，说的天花乱坠但没有实际证据。

解决方案：只讨论有 `run_id` 的实验。

周会模板

# 每周组会（30-45 分钟）

## 1. 轮流汇报（每人 5-10 分钟）

格式：

- 本周完成的实验：[列出 `run_id`]



- 主要发现：[基于实验数据，不是猜测]
- 遇到的问题：[具体问题，不是"不太好"]
- 下周计划：[具体目标，不是"继续调参"]

不接受的汇报：

- "我觉得这个方法应该有效"（没有实验支撑）
- "我跑了很多实验"（不列出 `run_id`）
- "效果还行"（不给出具体指标）

好的汇报示例：

"我跑了三个实验：

- 2026-02-15\_1030\_zhangsan\_baseline: 92.0%
- 2026-02-15\_1400\_zhangsan\_attention: 93.5% (+1.5%)
- 2026-02-16\_0900\_zhangsan\_attention\_v2: 93.2% (+1.2%)

结论：注意力机制有效，但版本 v2 引入的改进反而降低了性能。

下周计划：调查为什么 v2 更差，尝试修复。"

## ## 2. 团队决策（10 分钟）

- 哪些实验需要合并到主线？
- 哪些路径应该放弃？
- 下周谁负责什么？

## ## 3. 债务检查（5 分钟）

- 检查上周的 TODO 是否完成
- 检查是否有无记录的实验
- 检查是否有冲突的代码需要合并

## ## 4. 下周分工（5 分钟）

明确每个人的任务和交付物：

- 张三：完成消融实验（预计 3 个 run）
- 李四：改进数据增强（预计 2 个 run）
- 王五：整理论文图表脚本

## 10.5.2 异步沟通规范：用文档而非口头

问题：队友总在不合适的时间打断你问问题。

解决方案：建立”先看文档，再问人”的文化。

### 团队文档结构

```
docs/
 README.md # 项目总览
 SETUP.md # 环境配置指南
 WORKFLOW.md # 工作流程
 FAQ.md # 常见问题
 EXPERIMENTS.md # 实验追踪
 DECISIONS.md # 重要决策记录
 CONTACTS.md # 谁负责什么
```

### FAQ.md 示例

```
常见问题

Q: 如何配置环境?
见 SETUP.md

Q: 数据放在哪里?
~/data/shared/project_data_v3`
```

不要修改这个目录，只读。

## Q: 如何提交代码？

1. 创建分支: `git checkout -b exp/your-name-feature`
2. 完成改动并测试
3. 提交 PR (使用模板)
4. 等待 review
5. 合并后删除分支

## Q: 实验跑崩了怎么办？

1. 检查 `outputs/<run_id>/error.log`
2. 搜索 FAQ 看是否有类似问题
3. 如果找不到答案，在 issue 里提问（不要直接微信）

## Q: 如何复现别人的实验？

`make reproduce RUN=<run_id>`

## Q: 我可以改 `base.yaml` 吗？

不行。必须在团队会议讨论后才能改。

临时测试可以创建自己的 `config`。

## Q: 我的实验结果和别人不一样，怎么办？

1. 检查是否使用了相同的 `config`
2. 检查是否使用了相同的数据版本
3. 检查是否使用了相同的随机种子
4. 在周会上讨论

### 10.5.3 Issue 追踪：问题和任务可查询

问题：大家在微信群里讨论问题，三天后就找不到了。

解决方案：重要问题必须开 issue（GitHub/GitLab/Jira）。

## Issue 模板

# Bug Report

**\*\*问题描述\*\***：[简洁描述问题]

**\*\*复现步骤\*\***：

1. 运行命令：`python train.py --config ...`
2. 观察现象：[截图或日志]

**\*\*预期行为\*\***：[应该怎样]

**\*\*实际行为\*\***：[实际怎样]

**\*\*环境信息\*\***：

- 分支：[branch name]
- Commit：[commit hash]
- Python：[version]
- CUDA：[version]

**\*\*相关 run\_id\*\***：[如果有]

---

# Feature Request

**\*\*需求描述\*\***：[想要什么功能]

**\*\*使用场景\*\***：[为什么需要]

**\*\*建议实现\*\***：[可选]

---

# Task

**\*\*任务描述\*\***: [要做什么]

**\*\*验收标准\*\***: [怎样算完成]

**\*\*负责人\*\***: [@某人]

**\*\*截止时间\*\***: [YYYY-MM-DD]

**\*\*依赖\*\***: [依赖其他任务吗? ]

## 10.6 导师视角：如何带学生建立好习惯

### 10.6.1 第一天：设定期望

不要假设学生”自然会做对”。必须明确告诉他们你的期望。

#### 入职清单

# 新成员入职清单

## 第一天（环境配置）

- [ ] 配置开发环境（见 SETUP.md）
- [ ] 获得代码仓库权限
- [ ] 获得服务器访问权限
- [ ] 获得共享数据访问权限
- [ ] 阅读 README、WORKFLOW、FAQ

### ## 第一周（熟悉流程）

- [ ] 复现一个已有实验（验证环境配置正确）
- [ ] 跑一个小实验（验证理解流程）
- [ ] 完整记录实验（run.json + run.md）
- [ ] 提交第一个 PR（哪怕很小）

### ## 第一个月（独立工作）

- [ ] 独立完成一个探索方向
- [ ] 主动识别和解决问题
- [ ] 能够帮助新成员入职

## 10.6.2 定期 Review：不仅看结果，更看过程

不要只在周会上问”结果如何”，要检查”过程是否规范”。

### 每周 Code Review 清单

检查项：

- [ ] 本周的实验都有完整记录吗？
- [ ] Run ID 命名规范吗？
- [ ] Config 文件有更新吗？
- [ ] 代码提交 message 清晰吗？
- [ ] 有没有需要合并的改进？
- [ ] 有没有技术债需要处理？

如果发现问题，立刻指出并要求改正。不要”下不为例”，否则坏习惯会固化。

### 10.6.3 奖励好习惯

明确表扬做得好的地方：

- ”你这周的实验记录非常清晰，一眼就能看懂。”
- ”你的 PR 描述很详细，review 很顺畅。”
- ”你主动补充了文档，帮了大家的忙。”

让好习惯成为团队文化。

## 10.7 学生视角：如何在混乱的项目中生存

### 10.7.1 如果项目已经很混乱

你可能遇到：

- 导师的代码没有文档
- 师兄的代码跑不通
- 数据不知道放在哪里
- 没有人知道某个实验是怎么跑的

求生策略：

**策略 1：为自己建立”清洁区”**

# 在混乱项目里创建自己的子项目

```
my_workspace/
```

```
src/ # 你的代码（不依赖混乱部分）
configs/ # 你的配置
outputs/ # 你的实验记录
docs/ # 你的文档
README.md # 你的工作说明
```

即使整个项目很乱，至少你的部分是清晰的。

## 策略 2：写下你的理解

```
docs/MY_UNDERSTANDING.md
```

```
项目目标
```

```
[我理解的项目目标是什么]
```

```
现有代码
```

```
[我发现的关键文件和它们的作用]
```

```
已知问题
```

```
[我遇到的问题和临时解决方案]
```

```
我的工作
```

```
[我负责的部分和进展]
```

这份文档：

- 帮你理清思路
- 未来交接时有据可查
- 可以和导师同步理解



**策略 3：主动建立规范** 即使团队没有规范，你可以为自己建立规范：

- 你的所有实验都有 `run_id` 和记录
- 你的代码都有清晰注释
- 你的 `commit` 都有描述

好习惯会被注意到，可能会影响团队。

### 10.7.2 如何请求帮助

□ 糟糕的求助方式：

”师兄，我的代码跑不了，能帮我看看吗？”（没有任何上下文）

□ 好的求助方式：

师兄，我在复现 `baseline` 实验时遇到问题，能帮忙看看吗？

**\*\*问题\*\***：训练到第 10 个 `epoch` 时崩溃，报错 `CUDA out of memory`

**\*\*我的配置\*\***：

- 分支：`main`
- Commit：`a1b2c3d`
- Config：`configs/baseline.yaml`
- GPU：`V100 16GB`

**\*\*我尝试过的\*\***：

1. 减小 `batch size` 到 16（仍然崩溃）
2. 减小模型 `hidden_dim` 到 256（可以跑，但结果不对）

**\*\*日志\*\***: 见附件 `error.log`

**\*\*问题\*\***: 是否有其他配置需要调整？还是需要更大的 GPU？

清晰的求助信息会得到更快、更有用的回复。

## 10.8 冲突解决：常见团队问题

### 10.8.1 问题 1：代码风格不统一

解决方案：使用自动化工具强制统一。

```
安装 formatter 和 linter
pip install black flake8 isort

.pre-commit-config.yaml
repos:
 - repo: https://github.com/psf/black
 rev: 23.1.0
 hooks:
 - id: black

 - repo: https://github.com/PyCQA/flake8
 rev: 6.0.0
 hooks:
 - id: flake8

安装 pre-commit hook
pre-commit install
```

# 现在每次 `commit` 前会自动格式化

### 10.8.2 问题 2：实验结果对不上

排查清单：

1. [ ] 使用的代码版本相同？（`commit hash`）
2. [ ] 使用的配置相同？（`config diff`）
3. [ ] 使用的数据相同？（数据 `hash`）
4. [ ] 使用的随机种子相同？（`seed`）
5. [ ] 使用的环境相同？（Python/CUDA 版本）
6. [ ] 评估脚本相同？（`eval script`）

逐项检查，总能找到原因。

### 10.8.3 问题 3：分工不明确导致重复劳动

解决方案：用看板（Kanban）可视化任务。

# 可以用 GitHub Projects 或 Trello

看板列：

- 待做（TODO）
- 进行中（In Progress）
- 等待 Review（Review）
- 完成（Done）

每个任务卡片：

- 标题：[简短描述]
- 负责人：[@某人]

- 截止时间：[日期]
- 依赖：[依赖其他任务]
- 状态：[当前进展]

所有人都能看到”谁在做什么”，避免重复。

#### 10.8.4 问题 4：导师和学生期望不一致

解决方案：明确写下期望，定期对齐。

```
docs/EXPECTATIONS.md
```

##### ## 导师期望

- 每周至少 3 个有记录的实验
- 所有代码通过 lint 和测试
- 周会准时参加并汇报
- 遇到问题 24 小时内反馈

##### ## 学生期望

- 每周一次 1-on-1 答疑时间
- 关键决策会提前讨论
- 代码 review 48 小时内完成
- 论文截稿前一个月稳定需求

##### ## 双方承诺

- 诚实沟通，不隐瞒问题
- 尊重彼此时间
- 共同为项目成功负责

把期望显式化，可以避免很多隐性冲突。

## 10.9 10 分钟动作：建立团队第一个规范

如果你现在只做一件事：为团队建立第一个最小规范。

### 1. 如果你是导师/项目负责人（10 分钟）

```
创建团队规范文档
mkdir docs
cat > docs/TEAM_RULES.md <<EOF
团队最小规范

实验记录规范
- 每个实验必须有 run_id: YYYY-MM-DD_HHMM_name_experiment
- 每个实验必须有 run.json（自动）和 run.md（手动 5 行）

代码提交规范
- Commit message 格式：<type>: <description>
 - type: feat | fix | refactor | docs | test
- 提交前运行 make test

周会规范
- 每周一 10:00 固定时间
- 必须提前准备：列出 run_id 和主要发现
- 只讨论有实验证据的内容

沟通规范
- 重要问题开 GitHub issue（不要只在微信讨论）
- 紧急问题可以微信，但事后补 issue

生效时间
从下周一开始执行。
```

EOF

# 在下次组会宣布并讨论

## 2. 如果你是学生/团队成员（10 分钟）

# 为自己建立工作规范

```
cat > MY_WORKFLOW.md <<EOF
```

# 我的工作流程

## 每次实验后（5 分钟）

- [ ] 生成 run.json
- [ ] 填写 run.md 五要素
- [ ] Commit 相关代码改动

## 每周五（15 分钟）

- [ ] 整理本周实验列表
- [ ] 准备周会汇报材料
- [ ] 检查是否有遗漏的记录

## 每次提交代码前（2 分钟）

- [ ] 运行 make test
- [ ] 写清晰的 commit message
- [ ] 如果改动重要，开 PR

## 每次遇到问题时

- [ ] 先查 FAQ 和文档
- [ ] 如果找不到答案，开 issue（描述清晰）
- [ ] 问题解决后，更新 FAQ

EOF

# 从今天开始执行

### 3. 下次团队会议提议（准备 5 分钟发言）

建议在会议上说：

"我发现我们团队在实验记录和代码管理上有些混乱，  
建议我们建立一些最小规范，比如：

- 统一 `run_id` 格式
- 统一 `config` 管理方式
- 每周检查实验记录完整性

我草拟了一个初稿（见 `docs/TEAM_RULES.md`），  
大家可以讨论和补充。

建议从下周开始试行，一个月后评估效果。"

完成这个 10 分钟动作后，你会发现：

- 团队有了一个”起点”——从混乱到有序的起点
- 大家有了共同语言——知道什么是”做得好”
- 后续可以迭代改进——而不是一直混乱下去

## 10.10 本章总结：规范不是束缚，是高效协作的基础

团队协作的悖论：

- 没有规范：每个人都”自由”，但团队效率极低
- 过度规范：流程繁琐，束缚创造力
- 最小规范：刚好够用，既保证质量又保持灵活

最小规范的三个层次：

1. 代码规范：让代码可读、可维护、可协作
2. 实验规范：让实验可追溯、可对比、可复现
3. 沟通规范：让问题可查询、决策可追溯、分工可明确

记住：

- 规范不是”管理”工具，而是”协作”工具
- 规范不是”限制”，而是”减少摩擦”
- 规范不是”一次性建立”，而是”持续改进”

最后：如果你的团队现在很混乱，不要灰心。从一个最小规范开始，逐步改善。哪怕只有你一个人先做起来，好习惯也会慢慢影响团队。

高效团队不是”天生的”，而是”建立规范并持续执行出来的”。



# A 模板库（今天就能开始用）

---

## A.1 PR/提交说明模板（即使一个人也写）

- 目的：
- 改动：
- 影响结果？（是/否）：
- 验证命令：
- 预期输出：
- 风险点与回滚：

## A.2 实验记录 `run.md`（5 行就够）

- Hypothesis：
- Change：
- Result（metric + 结论一句话）：
- Compare-to（baseline run\_id）：
- Next：

### A.3 Makefile 目标（把复现变成习惯）

- `make test` (smoke test)
- `make train CONFIG=...`
- `make eval RUN=...`
- `make reproduce RUN=...`

## B AI 助手引发的真实研发“灾难”案例（用于对照）

---

本附录收集你提供的案例，并将它们抽象成可复用的工程教训。正文各章会穿插引用这些案例，作为“为什么要这样做”的现实证据。

### B.1 案例 A：用 Copilot 开发整套应用，最终推倒重写

现象：几乎全靠 AI 产码快速搭起应用，但随后发现每个部分都需要重写，最后提交代码里几乎不剩 AI 片段。

核心教训：AI 让你更快到达“能跑”，但不自动带来“可维护/可演进”。如果缺少测试、架构边界与验收标准，返工会被放大。

### B.2 案例 B：让 Copilot 包办代码库，反复返工且引入安全/逻辑隐患

现象：AI 能高效生成样板代码，但也会自信地产生错误假设（数据库结构、函数存在性、框架边界），甚至引入安全隐患；并可能过度设计，导致代码臃肿。

**核心教训：**AI 是放大器：你越不理解系统，它越会帮你更快失败；你越理解边界与数据流，它越能替你省时间。

### **B.3 案例 C：模型输出格式变动导致 workflow 崩溃**

**现象：**底层模型升级导致输出格式变化，下游解析/自动化步骤全部失效，被迫重写多个 workflow。

**核心教训：**把 AI 当成外部依赖接口来管理：需要版本意识、适配层、契约测试（contract test）。

### **B.4 案例 D：跨语言“翻译代码”看似完成，实际大量 API 幻觉**

**现象：**AI 生成的代码大量调用不存在的方法/错误用法，表面完整但无法运行，最终需要逐文件重写与修复。

**核心教训：**复杂迁移任务要以可执行测试驱动；“生成了很多文件”不等于完成。

### **B.5 案例 E：把项目完整托付 ChatGPT，20 天后拿不到任何交付**

**现象：**AI 口头承诺推进、上传、优化，但最终没有任何可用代码或报告产出。

**核心教训：**AI 不能替代项目管理与交付机制：必须把任务拆成可验收的最小增量，并要求每一步都有可运行产物。

## B.6 案例 F: ChatGPT 代写 ML 模型“漂亮地犯错”，险些造成重大损失

现象：离线指标很高，但因数据泄漏/类别不平衡等问题，线上预测荒谬，差点造成高额合同损失。

核心教训：研究与 ML 开发必须把验证债当头号风险：对照、泄漏检查、稳定性、线上/线下差异分析不可省。

## B.7 案例 G: AI 协同编程引发版本混乱与代码“消失”

现象：协同环境下出现文件回退、代码块消失、重复写三次功能，最终依赖 Git 历史找回。

核心教训：引入 AI 协作更需要严格的版本控制纪律：小步提交、频繁保存、可回滚。

## B.8 案例 H: ChatGPT 输出不可再现，科研复现性堪忧

现象：同样输入在不同时间得到不同输出，导致实验难以复现。

核心教训：把外部封闭模型当作“会漂移的实验仪器”：需要记录模型版本/时间戳/参数，并用稳健性评估替代单次输出结论。